



On the Optimization of Recursive Relational Queries: Application to Graph Queries

Louis Jachiet, Pierre Genevès, Nils Gesbert, Nabil Layaïda

► To cite this version:

Louis Jachiet, Pierre Genevès, Nils Gesbert, Nabil Layaïda. On the Optimization of Recursive Relational Queries: Application to Graph Queries. SIGMOD 2020 - ACM International Conference on Management of Data, Jun 2020, Portland, United States. pp.1-23, <10.1145/3318464.3380567>. <hal-01673025v6>

HAL Id: hal-01673025

<https://inria.hal.science/hal-01673025v6>

Submitted on 19 Jun 2021

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

On the Optimization of Recursive Relational Queries: Application to Graph Queries

Louis Jachiet

LTCI, Télécom Paris, Institut Polytechnique de Paris

Pierre Genevès

Nils Gesbert

Nabil Layaïda

Tyrex team, Univ. Grenoble Alpes, CNRS, Inria,
Grenoble INP, LIG, 38000 Grenoble, France

ABSTRACT

Graph databases have received a lot of attention as they are particularly useful in many applications such as social networks, life sciences and the semantic web. Various languages have emerged to query graph databases, many of which embed forms of recursion which reveal essential for navigating in graphs. The relational model has benefited from a huge body of research in the last half century and that is why many graph databases rely on techniques of relational query engines. Since its introduction, the relational model has seen various attempts to extend it with recursion and it is now possible to use recursion in several SQL or Datalog based database systems. The optimization of recursive queries remains, however, a challenge. We propose μ -RA, a variation of the Relational Algebra equipped with a fixpoint operator for expressing recursive relational queries. μ -RA can notably express unions of conjunctive regular path queries. Leveraging the fact that this fixpoint operator makes recursive terms more amenable to algebraic transformations, we propose new rewrite rules. These rules makes it possible to generate new query execution plans, that cannot be obtained with previous approaches. We present the syntax and semantics of μ -RA, and the rewriting rules that we specifically devised to tackle the optimization of recursive queries. We report on practical experiments that show that the newly generated plans can provide significant performance improvements for evaluating recursive queries over graphs.

This research has been partially supported by the ANR project CLEAR (ANR-16-CE25-0010).

ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.

SIGMOD'20, June 14–19, 2020, Portland, OR, USA

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-6735-6/20/06...\$15.00

<https://doi.org/10.1145/3318464.3380567>

ACM Reference Format:

Louis Jachiet, Pierre Genevès, Nils Gesbert, and Nabil Layaïda. 2020. On the Optimization of Recursive Relational Queries: Application to Graph Queries. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD'20)*, June 14–19, 2020, Portland, OR, USA. ACM, New York, NY, USA, 23 pages. <https://doi.org/10.1145/3318464.3380567>

1 INTRODUCTION

The expressive power of query languages has been greatly improved with the introduction of recursion. Recursive queries are, for instance, very useful in data integration since expressive ontologies use recursion [29]. Graph databases are another example where recursion is particularly useful for expressing navigation along paths connecting nodes in the graph. For this purpose, graph query languages often include constructs such as Regular Path Queries (RPQs) [27], and various extensions such as Conjunctions of them (CRPQs) and Union of CRPQs (UCRPQs) [19, 20, 26, 50]. For instance, the query language SPARQL 1.1 [43] introduced Property Paths, and language proposals such as OpenCypher [34, 55] and G-core [11] also include the possibility of expressing recursive paths. SPARQL's Property Paths revealed crucial for extracting information from RDF data structures such as those found in social networks, life sciences and transportation networks. However, recursive path queries are notoriously known to be much harder to optimize and evaluate than non-recursive ones [54, 71]. In practice, even with datasets of modest sizes, the benchmarking work found in [15] notices that “*all tested systems either failed on the majority of these [recursive] queries or had to be manually terminated after unexpectedly long running times.*” A major difficulty is to find an appropriate way to execute the query, a task frequently referred to in the literature as finding an appropriate Query Execution Plan (QEP). For example, let us consider the following query:

$?x$ isLocatedIn+ $?y$
Emmy_Noether worksAt $?x$ Q_{ex}

In a graph, this query retrieves all pairs of nodes $?x$, $?y$ such that Emmy Noether worked for $?x$ and $?x$ is located

in ?y, or is located in a place that is located in ?y, etc.. Formally, `isLocatedIn+` indicates that it is the *transitive closure* of `isLocatedIn`. Different QEPs exist for executing Q_{ex} . For instance:

- A first QEP, named $\mathcal{P}_1$, corresponds to computing first the transitive closure of `isLocatedIn` and then joining it with the ?x solution of `Emmy_Noether worksAt ?x`.
- Another QEP, noted $\mathcal{P}_2$, would rather start by computing the set of solutions for `Emmy_Noether worksAt ?x`. Then, these results are joined with the set of pairs ?x, ?y solutions of ?x `isLocatedIn` ?y. The resulting pairs of nodes are all solutions of Q_{ex} . Finally, iteratively, for each pair ?x, ?y in the set of solutions one can find the ?y' such that ?y `isLocatedIn` ?y' and add ?x, ?y' to the set of solutions.

The QEP $\mathcal{P}_2$ is generally more efficient than $\mathcal{P}_1$. Each pair of nodes (?x, ?y) processed in $\mathcal{P}_2$ is a solution of Q_{ex} . Therefore the total running time of $\mathcal{P}_2$ is linear in the number of solutions multiplied by the maximal degree of nodes in the graph. In contrast, in $\mathcal{P}_1$, the transitive closure of `isLocatedIn` always needs to be fully computed. This closure can contain, in the worst case, a number of elements which is quadratic in the number of `isLocatedIn`-labeled edges; and even in the best case, the transitive closure contains at least all the `isLocatedIn`-labeled edges. In the YAGO dataset [33, 64], that contains millions of entities and facts extracted from Wikipedia, there are only 16 solutions for Q_{ex} whereas there are millions of `isLocatedIn`-labeled edges. $\mathcal{P}_2$ is thus by far more efficient than $\mathcal{P}_1$.

Let us now consider a slightly more sophisticated example with the following query (taken from [5]), still intended to be executed against the YAGO dataset:

?x hasChild/livesIn/isLocatedIn+/dealsWith+ Japan Q_2

The variable ?x denotes any graph node representing a person. Q_2 retrieves all such nodes that are connected to a particular node labeled “Japan” through a path which must satisfy a regular path expression over edge labels in the graph. The regular path expression is recursive because it includes the subexpression $R=\text{isLocatedIn+}/\text{dealsWith+}$ in which the operator “+” stands for the transitive closure. Again, there are several ways to evaluate Q_2 for retrieving all possible values for ?x. In particular, there exists different QEPs for R corresponding to radically different manners of executing Q_2 . We briefly describe below three QEPs: $\mathcal{P}_3$, $\mathcal{P}_4$, $\mathcal{P}_5$ that we will use as examples:

- Plan $\mathcal{P}_3$ consists in first computing the two transitive closures `isLocatedIn+` and `dealsWith+` of the relations `isLocatedIn` and `dealsWith`, respectively, and then joining the results. One pitfall of this execution plan is that the sets to be joined might be very large (due to e.g. numerous locations in the database and the large size

of the set of pairs connected by “`isLocatedIn+`”), even though the overall query finally retrieves only few results.

- Early works on recursive query optimization [7, 8] already proposed how to push filters (and projections) as close to the sources as possible, even through recursive terms. Plan $\mathcal{P}_3$ can be optimized using this technique: the constant “Japan” can be “pushed inside the computation of `dealsWith+`”. This corresponds to plan $\mathcal{P}_4$ where the evaluation starts from ?t `dealsWith` Japan, then iteratively adds `dealsWith` steps on the left. Once this computation is finished, the results are joined with `isLocatedIn` and then, once again, iterative `isLocatedIn` steps are performed on the left. One advantage of $\mathcal{P}_4$ compared to $\mathcal{P}_3$ is that $\mathcal{P}_4$ processes each node at most twice while the whole transitive closures of `dealsWith` and `isLocatedIn` can be considerably larger (quadratic in the worst case).
- Another completely different way to evaluate R , noted plan $\mathcal{P}_5$, consists in not computing any transitive closure but, instead, computing first the composed relation “`isLocatedIn/dealsWith`” and then recursively navigating in the graph either hopping on the left with `isLocatedIn` or hopping on the right by `dealsWith` to retrieve nodes. Notice that this slightly more general form of recursion is not a transitive closure of any relation.

The plan $\mathcal{P}_3$ will generally be slower than $\mathcal{P}_4$ or $\mathcal{P}_5$, even when using smart algorithms for computing transitive closures [9, 21, 61]. On some instances $\mathcal{P}_4$ will be much more efficient (if, for instance, ?x `dealsWith+` Japan has few solutions) and on some other instances $\mathcal{P}_5$ will be faster (if, for instance, there are few solutions of `dealsWith/isLocatedIn`). This is the case when evaluating Q_2 over the YAGO graph (that has more than 62 million of edges): $\mathcal{P}_5$ is faster by a factor of more than 200x compared to other plans.

We investigate the problem of computing such query execution plans automatically.

Contribution. We introduce a theory, which is an extension of Codd’s classical relational algebra, for the purpose of automatically obtaining efficient QEPs for recursive queries. Specifically, we introduce a fixpoint operator “ μ ” in the relational algebra for denoting recursive terms in an algebraic manner. This fixpoint operator can express transitive closures as well as slightly more general forms of recursion. It makes recursive terms more amenable to transformations. We take advantage of this for introducing five new rewriting optimization rules. These rules allow generating new execution plans for recursive queries, that are beyond reach using previous approaches. We demonstrate empirically with

a prototype implementation that these new plans can provide significant performance gains in recursive graph query evaluation compared to previous approaches.

Outline. We first review related works on the topic in § 2. Then in § 3, we introduce the syntax and semantics of μ -RA: a variation of the relational algebra (RA) equipped with a fixpoint operator. § 4 describes properties of μ -RA and demonstrates that our fixpoint can be rewritten (to push filters, projections and joins inside of the fixpoints) and that two (or more) fixpoints can sometimes be merged into a unique fixpoint. We also explain why our approach is able to generate efficient plans that were beyond reach. As an application, we present how recursive graph queries translate into μ -RA in § 5. We then report on practical experiments in § 6, in which we benchmark a prototype implementation with state-of-the-art systems for evaluating recursive graph queries. For reading purposes, we present only proof sketches of our main theorems; the full proofs being available in [12].

2 RELATED WORK

We first present the main approaches that have been proposed to evaluate recursive queries, and we explain why our approach can be more efficient, in particular for the specific kind of recursive queries expressed as UCRPQs.

2.1 The relational model

The relational model [25] introduced by Codd in 1970 has become the de facto paradigm for querying data banks. The design of the prevailing database query language SQL has been heavily influenced by the Relational Algebra (RA).

One of the main interest of the RA (and of SQL) is that it allows programmers to express the data they are interested in without specifying the way to retrieve it [51], instead they rely on a relational query engine to find an efficient way to process queries. Most relational query engines thus rely on a optimization process where the query to be processed is translated into an algebraic term. Then, this term is rewritten into a set of semantically equivalent terms (by e.g. pushing selections, or projections as close as possible to the sources). These different terms can be used to generate different QEPs. For a given query, the set of QEPs is called a *Plan Space*. The plan space depends on the set of equivalent terms that we can generate from the initial query.

2.2 Recursive queries & expressive power

Soon after the introduction of RA, the work found in [8] noticed that the RA lacks the possibility of expressing recursive queries. Several formalisms have been introduced as attempts to fill this gap. We now briefly review the most closely related of these various formalisms (see e.g. [2, 18] for complete surveys).

The α -extended RA [7] extends the RA with a recursive facility noted α . The operator α is a transitive closure operator; given an α -extended RA term R defining a binary relation $\mathcal{R}$, $\alpha(R)$ represents its transitive closure, $\mathcal{R}^+$. In terms of expressive power over graphs, this corresponds to UCRPQs. It is for instance not capable of querying the path expression a^n/b^n which queries paths whose labels are first a s then b s with exactly the same number of a and b .

A more powerful way of extending the RA is the LFP-RA. LFP-RA is the RA extended with a “least fixpoint” construct. Given a LFP-RA term R parametric in a relation X , the *least fixpoint* of R over X , noted $\mu(X = R)$, is obtained as the limit of a sequence $(X_i)_{i \in \mathbb{N}}$ where $X_0 = \emptyset$ and X_{i+1} is computed by adding to X_i the results of evaluating R when the relation X is filled with X_i .

This LFP-RA formalism has the same expressive power and the same data complexity as Datalog with stratified negation, for which evaluation is known to lie in PTIME.

Because LFP-RA terms can be computationally hard to evaluate, several syntactic fragments of LFP-RA have been studied. A well-studied fragment of this LFP-RA corresponds to limiting fixpoints so that all recursions are “linear” (in a sense that we will define in Definition 6). This restriction makes the expressive power drop to linear Datalog [6], which is strictly between the expressive power of α -extended RA and the expressive power of Datalog. For instance, all UCRPQs as well as the non-regular path expression a^n/b^n are expressible in linear Datalog; however, the path expression asking for all paths containing exactly as many a labels as b labels *in any order* is not. *The μ -RA that we use in this paper is a variant of this restricted LFP-RA and it has the same expressive power as linear Datalog.*

Another way of extending the RA is the WHILE language (see [2]). This language is known to be at least as expressive as LFP-RA and the inclusion is strict (unless PTIME=PSpace).

2.3 Optimization of recursive RA

In 1979, a first line of work [8] proposed a restricted LFP-RA. The authors showed that some optimizations can be performed on fixpoints, such as pushing selections into these fixpoints (similar to our rule RW1). One drawback of their method is that it has more restrictions on fixpoints than μ -RA. Being too restricted makes [8] unable to express some of our rewritten terms, e.g. merged fixpoints (see § 4.1.3). Furthermore, even if the authors note that it is sometimes possible to push projections (similar to our rule RW5) they do not provide an effective criterion for this.

Using the (unrestricted) LFP-RA, other authors [46] provided in 1990 a general framework for optimization. Their method works on so-called *system graphs* and computes a fixpoint of filters that is safe to apply recursively. Their work

also provides an effective criterion to push projections. We believe our approach is more straightforward. In addition, they do not deal with conjunctions (our rules RW3 and RW4) and thus cannot find plan $\mathcal{P}_2$ for Q_{ex} nor find plan $\mathcal{P}_5$ for Q_2 , for instance. Finally, compared to unrestricted LFP-RA, μ -RA comes with restrictions on fixpoints that are needed for the validity of the rewrite rules we propose in § 4.1: the optimizations we present would not be correct on the unrestricted LFP-RA (as explained in § 3.4).

2.4 Datalog

The term Datalog has been coined at the end of the 70s to designate the fragment of logic programming restrained to data. Datalog is a language supporting recursive queries. Many Datalog-based query evaluators exist [13, 28, 49, 57, 68] including recent fully-fledged commercial systems [13].

2.4.1 Magic Sets. A well-known Datalog optimization technique is the “Magic Sets” algorithm (see [17, 58]). The syntax of Datalog vastly differs from the RA, but the effect of Magic Sets algorithm is very similar to pushing some type of selections and projections. The idea of the magic set is to compute, for each datalog relation, the set of “contexts” where this term will be evaluated. For instance, in the translation of `?a dealsWith+ Japan`, the magic set method can sometimes detect that, on the recursive use of `dealsWith`, the right side is always Japan, and it will not compute the full transitive closure `dealsWith+`.

2.4.2 Right and left linear programs. There are many different ways to translate transitive closures in Datalog. For the transitive closure R^+ , one translation would use that R^+ is either a path composed of a single R or the concatenation of a path R and a path R^+ . Such a translation is called right linear because R^+ is computed by adding R paths on the left. Another translation would be left-linear: R^+ is then either a path R or the concatenation of R^+ with R . As noticed before [53], given a Datalog term t computing a binary relation $P(x, y)$, the Magic Set algorithm is able to push filters on the right side (the y) only if t is a right-linear program; and conversely, it can only push filters on the left side (the x) when t is left-linear. The authors of [53] thus proposed an automated way to “reverse” right-linear programs into left-linear (and vice-versa). This reversal can then be used by a query optimizer in combination with the Magic Set algorithm.

2.4.3 Demand Transformation. The Demand-driven Transformation, or Demand Transformation, is a recent improvement [65] over the Magic Sets. The idea is similar to the Magic Set: it pushes filters to avoid computing some “useless” facts. It has been proved that Demand Transformation always beats Magic Sets; however, it still suffers from some

of the problems of Magic Sets. In particular, Demand Transformation is also sensitive to whether programs are left or right linear; and on examples containing the concatenation of two transitive closures (such as `isLocatedIn+/dealsWith+` in Q_2), the query execution plan computing first the concatenation `isLocatedIn/dealsWith` and then recursively adding `isLocatedIn` on the left or `dealsWith` on the right will not be found by the Datalog engine, even after Demand Transformation and program reversals.

2.4.4 Overall Comparison. Datalog engines do not explore Plan Spaces but use heuristics to find a good plan to evaluate queries. Depending on which combination of Magic Sets, Demand Transformation and Reversals it uses, a Datalog engine might be able to find a plan similar to $\mathcal{P}_2$ for Q_{ex} , and to plan $\mathcal{P}_4$ for Q_2 . Since none of these optimizations can “merge” recursive terms, they do not allow to find a plan similar to $\mathcal{P}_5$ for Q_2 , which is –by an order of magnitude– the fastest plan for Q_2 on YAGO.

In a Datalog program corresponding to the optimized translation of $A+/B+$ at least one of the two transitive closures will be fully materialized (even if there are 0 solutions to $A+/B+$).

2.5 SQL

Since its 1999 version, the SQL standard supports recursive queries. SQL is more expressive than our μ -RA as it allows, e.g., arithmetic and aggregation, but if we restrict SQL to its core our proposed μ -RA is not very different from SQL with recursion. However, a restriction in SQL (the recursion variable cannot appear more than once in the recursive part of the query) forbids what would be the literal translation of some μ -RA terms; in particular, we cannot translate the merged fixpoint of rule RW4 into standard SQL. Furthermore, recursive queries in SQL are not supported by all vendors and those who do support them tend to consider them as optimization barriers. There are exceptions (such as DB2) but these vendors use a technique inspired by the Magic Set technique invented for Datalog.

2.6 Ad-hoc evaluation of UCRPQs

Up to now, we have only compared systems operating on languages strictly more expressive than UCRPQs. If we are only interested in UCRPQs or RPQs there are others systems and we now present the main ones.

2.6.1 Automata. One way to evaluate UCRPQ is to translate individual RPQs to automata, run them, and then union-join the individual results. This automata-based method is clearly not optimal for UCRPQs as the various RPQs are considered individually and therefore the constraints on one RPQ cannot be used on the others but the automata

is sometimes not optimal even on a single RPQ. Indeed, let us suppose that we are considering the regular expression $(a/b/c)^+$, the automata approach starts by computing the solution to the path a , then to the path a/b , then $a/b/c$ and recursively restarts (i.e. in a computational form we have $R_{i+1} = ((R_i/a)/b)/c$). This method forces the associativity of the computation [70] and does not test some associations such as computing $R_{i+1} = (R_i/a)/(b/c)$ in which we precompute paths matching (b/c) and which can, in some cases, be much more efficient, when e.g. there are only a handful of solutions to the path b/c .

2.6.2 α -extended RA. The translation of the RPQ $(a/b/c)^+$ in the α -extended RA includes a term $\alpha(t)$ where t contains $(a/b/c)$. Therefore the relation $(a/b/c)$ will first be fully computed, and then its transitive closure. This is not always optimal, as it suffers from the same drawbacks as the previous approach.

2.6.3 Waveguide. The Waveguide paper [70] introduced a new technique to evaluate RPQs that mixes ideas from automata and from the α -extended RA. Their idea is that the “interesting” plans to evaluate one RPQ can either start on the left of the RPQ and try to match the right part, or do the opposite (start on the right part and match the left one) or start in the middle and go both ways. For instance on Q_2 they will try all the plans that we express. However, since they focus on a single RPQ, for a conjunction of RPQs such as Q_{ex} Waveguide will not take advantage of the constraint on $?x$ and will materialize the full relation $owns^+$. Moreover, on a query $?a\ dealsWith^+ ?b, ?b\ isLocatedIn^+ ?c$, our approach will have a single fixpoint in which the number of mappings treated is exactly the number of solutions, while their approach will compute and then join the full transitive closures $dealsWith^+$ and $isLocatedIn^+$.

The authors of Waveguide extended their work in two short papers. The first one describes Tasweet, a system focusing on disjunction of RPQs and the second one presents Wireframe, a tool focusing on conjunctions of RPQs. Wireframe computes a “query spanning tree” to decide in which order the RPQs should be evaluated, and then it relies on Waveguide for individual RPQs. Tasweet improves on Waveguide by noticing that given a set of disjunctive RPQs, some computations can be shared. Both of these works suffer from the limitations of Waveguide. For instance, in Wireframe the constraints on one node can be used for the evaluation of other RPQs but the evaluation of various RPQs cannot be interleaved. Furthermore, these tools are limited to conjunctions or unions of RPQs.

3 THE μ -EXTENDED REL. ALGEBRA

We now present the μ -extended relational algebra, or μ -RA, which is our variation of the domain-independent relational algebra, equipped with a fixpoint. We first recall some usual definitions, and then present its syntax, types and semantics.

The formalism introduced in this section is classical (but included for self-completeness) except for the anti-projection and the fixpoint operator. Anti-projection replaces the more common projection operator. Anti-projections and the restrictions on fixpoints will allow us to introduce powerful rewrite rules in the next section.

3.1 Data model

Our data model is the same as for the classical relational algebra: we consider *relations* which are sets of *mappings* (also called tuples, or lines) which associate *column names* to *values*. Formally, we assume the following constants:

- $\mathbb{V}$ an infinite set of *values*;
- $\mathbb{C}$ an infinite set of *column names*;
- $\mathbb{R}$ an infinite set of *relation names*.

DEFINITION 1. A mapping or tuple is a partial function $m: \mathbb{C} \rightarrow \mathbb{V}$ whose domain is finite. If $\text{dom}(m) = \{c_1, \dots, c_n\}$, m can also be seen as the set $\{c_1 \rightarrow m(c_1), \dots, c_n \rightarrow m(c_n)\}$.

DEFINITION 2. Two mappings m_1 and m_2 are compatible, noted $m_1 \sim m_2$, when $\forall c \in \text{dom}(m_1) \cap \text{dom}(m_2), m_1(c) = m_2(c)$. If m_1 and m_2 are compatible, we define $m_1 + m_2 : \text{dom}(m_1) \cup \text{dom}(m_2) \rightarrow \mathbb{V}$ by:

$$(m_1 + m_2)(c) = \begin{cases} m_1(c) & \text{if } c \in \text{dom}(m_1) \\ m_2(c) & \text{if } c \in \text{dom}(m_2) \end{cases}$$

If we see mappings as sets, this corresponds to their union.

DEFINITION 3. A relation is a finite set of mappings which share the same domain. We call this common domain the type of the relation. We do not consider datatypes (all values are in the single domain $\mathbb{V}$): for simplicity, a type is just a set of column names. The empty relation is considered compatible with all types.

Relations represent data. A relational database is a finite set of named relations (also called tables). We represent such a database as a triple $(\mathcal{R}, \Gamma, D)$ where: $\mathcal{R} \subset \mathbb{R}$ is the set of relation names; Γ , the database schema, associates relation names to relation types; and D , the database body, associates relation names to actual relations. The body must be consistent with the schema: for any $R \in \mathcal{R}$, $D(R)$ has type $\Gamma(R)$.

3.2 Syntax of μ -RA terms

Our algebra μ -RA is mainly a variation of the relational algebra, with the addition of a fixpoint operator μ ; it operates on relations. The terms represent queries and are built from

$$\begin{aligned}
\llbracket \varphi_1 \bowtie \varphi_2 \rrbracket_V &= \{m_1 + m_2 \mid m_1 \in \llbracket \varphi_1 \rrbracket_V \wedge m_2 \in \llbracket \varphi_2 \rrbracket_V \wedge m_1 \sim m_2\} & \llbracket \varphi_1 \cup \varphi_2 \rrbracket_V &= \llbracket \varphi_1 \rrbracket_V \cup \llbracket \varphi_2 \rrbracket_V \\
\llbracket \varphi_1 \triangleright \varphi_2 \rrbracket_V &= \{m \in \llbracket \varphi_1 \rrbracket_V \mid \forall m' \in \llbracket \varphi_2 \rrbracket_V \neg(m' \sim m)\} & \llbracket |c \rightarrow v| \rrbracket_V &= \{\{c \rightarrow v\}\} \\
\llbracket \tilde{\pi}_a(\varphi) \rrbracket_V &= \left\{ \{c \rightarrow v \in m \mid c \neq a\} \mid m \in \llbracket \varphi \rrbracket_V \right\} & \llbracket X \rrbracket_V &= V(X) \\
\llbracket \rho_a^b(\varphi) \rrbracket_V &= \left\{ \{c \rightarrow v \in m \mid c \neq a\} \cup \{b \rightarrow v \mid a \rightarrow v \in m\} \mid m \in \llbracket \varphi \rrbracket_V \right\} & \llbracket \sigma_{\mathfrak{f}}(\varphi) \rrbracket_V &= \{m \mid m \in \llbracket \varphi \rrbracket_V \wedge \mathfrak{f}(m) = \top\} \\
\llbracket \mu(X = \varphi) \rrbracket_V &= \llbracket X \rrbracket_{V[X/U_\infty]} \quad \text{where } U_0 = \emptyset, U_{i+1} = U_i \cup \llbracket \varphi \rrbracket_{V[X/U_i]}, \text{ and } U_\infty = \bigcup_{n \in \mathbb{N}} U_i
\end{aligned}$$

Figure 1: Semantics of μ -RA

$\varphi ::=$	term
X	relation variable
$ c \rightarrow v $	constant
$\varphi_1 \cup \varphi_2$	union
$\varphi_1 \bowtie \varphi_2$	join
$\varphi_1 \triangleright \varphi_2$	antijoin
$\sigma_{\mathfrak{f}}(\varphi)$	filtering
$\rho_a^b(\varphi)$	renaming
$\tilde{\pi}_a(\varphi)$	anti-projection
$\mu(X = \varphi)$	fixpoint

Figure 2: Grammar of μ -RA

relation variables and operations; given a mapping from variables to relations (representing a database body), a term can be evaluated and yields another relation (the solution of the query). Evaluation of terms is described in § 3.3.

3.2.1 Filters. The standard selection operation $\sigma_{\mathfrak{f}}$, which operates on a relation by keeping only a subset of its mappings, depends on a *filter* $\mathfrak{f}$ indicating which mappings are to be kept. This filter can be seen as a function from mappings to booleans. To keep things focused, we do not detail here a syntax for filters, but we assume that for any filter $\mathfrak{f}$ we can compute a set $FC(\mathfrak{f})$ of column names such that the result of $\mathfrak{f}(m)$ depends only on $\{c \rightarrow m(c) \mid c \in FC(\mathfrak{f})\}$.

3.2.2 Terms. The core syntax of terms is defined in Fig. 2. The base terms are relation variables X and constants $|c \rightarrow v|$ (representing a single mapping with a singleton domain). Two relations can be combined with the classical relational operators $\cup$, $\bowtie$ and $\triangleright$. One relation can be filtered using the classical selection operation $\sigma_{\mathfrak{f}}$ where $\mathfrak{f}$ is a filter. The rename operator $\rho_a^b(\cdot)$ renames the column a into b . Less classically, the anti-projection $\tilde{\pi}_a(\cdot)$ (or column dropping) removes column a . The projection operator $\pi_{p_1, \dots, p_n}(\varphi)$ can be expressed in terms of $\tilde{\pi}(\cdot)$ provided we know the type of φ : if φ has type $t = \{p_1, \dots, p_n, a_1, \dots, a_k\}$ we have $\pi_{p_1, \dots, p_n}(\varphi)$ equivalent to $\tilde{\pi}_{a_1}(\dots \tilde{\pi}_{a_k}(\varphi))$. Our choice of anti-projection will allow us to extend the domains of subterms without changing the

projections, as in $\tilde{\pi}_a(\varphi) \bowtie \psi \rightarrow \tilde{\pi}_a(\varphi \bowtie \psi)$ when a is not in the type of ψ .

Finally, we introduce the fixpoint term $\mu(X = \varphi)$ representing a recursive query. In this term, there are some additional restrictions on φ , which will be detailed in § 3.4. The result R of this operation is a fixpoint in the sense that evaluating φ with X bound to R must yield R again. The restrictions we add ensure that this fixpoint exists and can be computed iteratively. We consider μ as a variable binder, yielding the standard notions of *free* and *bound* variable occurrences:

DEFINITION 4. In a term φ , all occurrences of a variable X which appear in a subterm of the form $\mu(X = \psi)$ are bound. All other occurrences of X are free.

As will be clear from the semantics, bound variables can be renamed, as usual, without changing the meanings of the terms. We can thus assume for simplicity that all bound variables are different from each other and from free variables.

3.3 Semantics

In μ -RA, relation variables X are used to denote both references to a database relation and a recursive relation. In a full query, the two are distinguished by the fact that database references appear as free variables, whereas recursion variables are bound by μ ; but in a subterm, we do not need to distinguish the two. In all cases, the semantics of a term φ depends on an *environment* V which maps all free variables of φ to relations.

The semantics is defined in Fig. 1, where $\llbracket \varphi \rrbracket_V$ designates the result of evaluating φ in the environment V . This result is defined recursively from the results of evaluating the subterms. The initial environment for evaluating the whole term is a database body D , but in evaluating $\mu(X = \varphi)$, the recursive calls use different environments where the recursion variable X is given a value: the notation $V[X/S]$ represents the environment V altered by mapping X to S .

3.4 Restrictions on fixpoints

Our syntax for the μ -RA is very general and comprises some counter-intuitive fixpoints and some types of fixpoints that

are hard to optimize (e.g. non linear & mutually recursive). We present restrictions on fixpoints that are needed for the validity of our rewrite rules and of most of our propositions and theorems.

In the sequel, we suppose that all fixpoints abide the restrictions that we present here. This does not mean, however, that our method can not be applied on general terms: given a general term φ that contains a subterm ψ , if ψ abides the restrictions then we can apply our rewrite rules on ψ .

3.4.1 Properties of fixpoints.

DEFINITION 5. Given a term φ , we say that φ is constant in X when X is not a free variable of φ .

DEFINITION 6. A fixpoint term $\mu(X = \varphi)$ is said:

- positive when for all subterms $\varphi_1 \triangleright \varphi_2$ of φ , φ_2 is constant in X ;
- linear when for all subterms of φ of the form $\varphi_1 \bowtie \varphi_2$ or $\varphi_1 \triangleright \varphi_2$, either φ_1 or φ_2 is constant in X ;
- mutually recursive when there exists a subterm $\mu(Y = \psi)$ of φ with X free in ψ .

PROPOSITION 1. If $\mu(X = \varphi)$ is linear, positive and non mutually recursive then the function $f(S) = \llbracket \varphi \rrbracket_{V[X/S]}$ is such that:

$$f(S) = f(\emptyset) \cup \bigcup_{x \in S} f(\{x\})$$

and thus f has a fixpoint with $\llbracket \mu(X = \varphi) \rrbracket_V = f^\infty(\emptyset)$.

PROOF SKETCH. We prove by induction on the subterms ξ of φ where X is free in ξ that $\llbracket \xi \rrbracket_{V[X/S]} = \llbracket \xi \rrbracket_{V[X/\emptyset]} \cup \bigcup_{x \in S} \llbracket \xi \rrbracket_{V[X/\{x\}]}$. By linearity, such a ξ can only be combined with a constant term and by positivity, it cannot be negated. $\square$

3.4.2 Expressivity of restricted fixpoints. These restrictions do have an effect on the expressivity of our language. We can show that μ -RA has, at least, the expressive power of inflationary-Datalog⁻ (Datalog with inflationary semantics and negation). When we restrict fixpoints to be positive and non mutually recursive then our language has exactly the expressive power of stratified-Datalog⁻. Finally with all our restrictions, our language has exactly the expressive power of linear datalog (see § 2.2). This fragment is expressive enough to capture a lot of interesting queries. For instance, the next section presents how to translate UCRPQs into μ -RA (with the restrictions).

As mentioned previously, our method can be applied on general terms but, for the sake of simplicity, in the sequel we will only consider the fragment *rest- μ -RA* of μ -RA containing only terms where all the fixpoints are **linear, positive and non mutually recursive**.

3.5 Decomposed fixpoints

Once our terms are restricted, we see that fixpoints can actually be decomposed into a strictly *recursive* part and a *constant* part. This decomposition will be later useful for expressing some of our rewrite rules.

DEFINITION 7. Given a term φ linear and positive in X , we say that φ is recursive in X when $\text{rec}(\varphi, X) = \top$ with rec defined as:

$$\begin{aligned} \text{rec}(\varphi_1 \cup \varphi_2, X) &= \text{rec}(\varphi_1, X) \wedge \text{rec}(\varphi_2, X) \\ \text{rec}(\varphi_1 \bowtie \varphi_2, X) &= \text{rec}(\varphi_1, X) \vee \text{rec}(\varphi_2, X) \\ \text{rec}(\varphi_1 \triangleright \varphi_2, X) &= \text{rec}(\varphi_1, X) \\ \text{rec}(\sigma_{\bar{t}}(\varphi), X) &= \text{rec}(\varphi, X) \\ \text{rec}(\pi_a(\varphi), X) &= \text{rec}(\varphi, X) \\ \text{rec}(\rho_a^b(\varphi), X) &= \text{rec}(\varphi, X) \\ \text{rec}(\mu(Y = \varphi), X) &= \perp \\ \text{rec}(X, Y) &= X = Y \\ \text{rec}(|c \rightarrow v|, X) &= \perp \end{aligned}$$

Being recursive or constant (def. 5) are syntactical properties. However the two following propositions give a semantic interpretation of those syntactical properties.

LEMMA 1. Let φ be a term.

- If φ is recursive in X then for all V , $\llbracket \varphi \rrbracket_{V[X/\emptyset]} = \emptyset$.
- If φ is constant in X , then φ does not depend on X , i.e. for all S and V , $\llbracket \varphi \rrbracket_{V[X/S]} = \llbracket \varphi \rrbracket_{V[X/\emptyset]}$.

DEFINITION 8. A fixpoint term $\mu(X = \kappa \cup \psi)$ is said decomposed when κ is constant in X and ψ is recursive in X .

EXAMPLE 1. Let us suppose that we want to compute the transitive closure of a binary relation R . We assume R is represented as a two-column table of type $\{\text{src}, \text{trg}\}$. The closure is captured by the term $\mu(X = R \cup \pi_m(\rho_{\text{trg}}^m(R) \bowtie \rho_{\text{src}}^m(X)))$.

This term is a decomposed fixpoint: R is constant and $\pi_m(\rho_{\text{trg}}^m(R) \bowtie \rho_{\text{src}}^m(X))$ is recursive in X .

PROPOSITION 2. A fixpoint term $\mu(X = \varphi)$, linear, positive and non mutually recursive can be rewritten to either: an empty term, a term φ with one less fixpoint or a decomposed fixpoint.

In the following, we assume that fixpoints are always decomposed.

3.6 Type System

Given a schema Γ for a set of relation variables $\mathcal{R}$, we can infer types for terms whose free variables are in $\mathcal{R}$. The typing judgement $\Gamma \vdash \varphi : t$ means that when evaluated in an environment conforming to the schema Γ , φ will yield a relation of type t . Our type system is defined by the rules on Fig. 3; it is quite straightforward. The rule for typing fixpoints uses the fact that fixpoints are decomposed to infer first the type of the constant part and then use that information to typecheck the recursive part. Given a database schema Γ ,

we write $\mathcal{F}[\Gamma]$ for the set of well-typed terms in Γ (i. e. the terms φ such that $\Gamma \vdash \varphi : t$ holds for some t).

PROPOSITION 3. *Given a database $(\mathcal{R}, \Gamma, D)$ and $\varphi \in \mathcal{F}[\Gamma]$, if $\Gamma \vdash \varphi : t$ then the relation $\llbracket \varphi \rrbracket_D$ has type t .*

EXAMPLE 2. *Consider again the term of Example 1: $\mu(X = R \cup \tilde{\pi}_m(\rho_{\text{trg}}^m(R) \bowtie \rho_{\text{src}}^m(X)))$. Assuming R is of type $\{\text{src}, \text{trg}\}$, we can conclude the whole term is also of type $\{\text{src}, \text{trg}\}$.*

Indeed, the fixpoint should have the type of its constant part, which is R . Then we can check the type of the recursive part: $\rho_{\text{trg}}^m(R)$ has type $\{\text{src}, m\}$ and is joined with $\rho_{\text{src}}^m(X)$ of type $\{\text{trg}, m\}$. The result has type $\{\text{src}, \text{trg}, m\}$ but the m column is discarded by the $\tilde{\pi}_m(\dots)$; thus the whole term is well typed.

4 GENERATING NEW QUERY PLANS

The traditional RA has rewrite rules and the optimization of relational queries is usually done by rewriting to a (estimated) more efficient term using rewrite rules. For instance the rule:

$$\sigma_{\bar{f}}(\varphi \bowtie \psi) \rightarrow \sigma_{\bar{f}}(\varphi) \bowtie \psi$$

describes that a filter can be pushed inside a join. This rule applies whenever $FC(\bar{f}) \subseteq t$ where t is the type of φ .

In this section, we discuss properties of *rest-μ-RA* which allow us to introduce new rewrite rules for recursive terms. These rules extend the set of RA classical rewrite rules, which remain valid on *rest-μ-RA*. We first describe the new rules informally and briefly explain how they can lead to terms that are more efficiently evaluated. Then in § 4.2 we introduce the “set of derivations” for a term which allows us to describe when the two first rules are correct. Finally, in § 4.3 we introduce “addable columns” which allow us to introduce the conditions for three other rewrite rules to apply.

4.1 The new rewrite rules

4.1.1 Pushing filters and anti-join into fixpoints:

$$\sigma_{\bar{f}}(\mu(X = \varphi)) \rightarrow \mu(X = \sigma_{\bar{f}}(\varphi)) \quad (\text{RW1})$$

$$\mu(X = \varphi) \triangleright \psi \rightarrow \mu(X = \varphi \triangleright \psi) \quad (\text{RW2})$$

This always reduces the amount of mappings manipulated by the fixpoint. For example, on the RPQ $?x \text{ isLocatedIn+ Japan}$, the first rule permits to only compute the $?x$ where $?x \text{ isLocatedIn+ Japan}$, instead of computing the whole transitive closure isLocatedIn+ and then filtering out the pairs not in Japan.

4.1.2 Pushing joins into fixpoints:

$$\mu(X = \varphi) \bowtie \psi \rightarrow \mu(X = \varphi \bowtie \psi) \quad (\text{RW3})$$

This can also decrease the number of mappings solutions of the fixpoint. This rewrite rule can typically be used on RPQs such as Q_{ex} . In this query, there is no need

to compute the whole set of pairs $(?x, ?y)$ such that $?x \text{ isLocatedIn+ ?y}$, as only the $?x$ such that $\text{Emmy_Noether worksAt ?x}$ are of interest to us. Rule RW3 can be used here to push $\text{Emmy_Noether worksAt ?x}$ into the fixpoint of $?x \text{ isLocatedIn+ ?y}$.

4.1.3 Merging fixpoints:

$$\mu(X = \kappa \cup \psi) \bowtie \mu(X = \kappa' \cup \psi') \rightarrow \mu(X = \kappa \bowtie \kappa' \cup \psi \cup \psi') \quad (\text{RW4})$$

This reduces the number of fixpoints and the number of mappings. For instance, on Q_2 : instead of computing isLocatedIn+ and dealsWith+ separately and then joining them, rule RW4 permits to first compute $\text{isLocatedIn/dealsWith}$ and then to add steps isLocatedIn on the left or steps dealsWith on the right.

4.1.4 Pushing antiprojections into fixpoints:

$$\tilde{\pi}_p(\mu(X = \varphi)) \rightarrow \mu(X = \tilde{\pi}_p(\varphi)) \quad (\text{RW5})$$

This can also reduce the number of mappings: the value of the removed column being ignored, several mappings might get merged (due to set semantics). This is typically useful for RPQs in which a fixpoint is joined or filtered and the value of the join or filter variable is discarded.

We now discuss formally the conditions which allow these rewritings. All the proofs are in [12].

4.2 Set of derivations of a term

In general, we do not have $\sigma_{\bar{f}}(\mu(X = \varphi)) \equiv \mu(X = \sigma_{\bar{f}}(\varphi))$. Indeed, it is possible that some mappings solution of $\mu(X = \varphi)$ do not pass the filter but are still useful to create mappings (with the fixpoint iteration) which do pass the filter.

To rule out this possibility, we can use the following criterion: if the filter depends only on columns which are untouched by the fixpoint iteration, then applying the filter before or after the iteration is equivalent. To check this criterion syntactically, we introduce the notion of *derivation*, which describes which columns in the result tuples depend on which columns in the initial ones. This then allows us to define the *stabilizer* of a fixpoint, the set of columns which are untouched during the iteration.

4.2.1 Logical framework.

DEFINITION 9. *The set of derivations $d(\varphi, X)$ is:*

$\Gamma \vdash c \rightarrow v : c$	$\frac{\Gamma(X) = t}{\Gamma \vdash X : t}$	$\frac{\Gamma \vdash \varphi_1 : t \quad \Gamma \vdash \varphi_2 : t}{\Gamma \vdash \varphi_1 \cup \varphi_2 : t}$	$\frac{\Gamma \vdash \varphi_1 : t_1 \quad \Gamma \vdash \varphi_2 : t_2}{\Gamma \vdash \varphi_1 \bowtie \varphi_2 : t_1 \cup t_2}$	$\frac{\Gamma \vdash \varphi_1 : t_1 \quad \Gamma \vdash \varphi_2 : _}{\Gamma \vdash \varphi_1 \triangleright \varphi_2 : t_1}$
$\frac{\Gamma \vdash \varphi : t \quad FC(\mathfrak{f}) \subseteq t}{\Gamma \vdash \sigma_{\mathfrak{f}}(\varphi) : t}$	$\frac{\Gamma \vdash \varphi : t \quad a \in t \quad b \notin t}{\Gamma \vdash \rho_a^b(\varphi) : (t \setminus \{a\}) \cup \{b\}}$	$\frac{\Gamma \vdash \varphi : t \quad a \in t}{\Gamma \vdash \tilde{\pi}_a(\varphi) : t \setminus \{a\}}$	$\frac{\Gamma \vdash \kappa : t \quad \Gamma \cup \{X \rightarrow t\} \vdash \psi : t}{\Gamma \vdash \mu(X = \kappa \cup \psi) : t}$	

Figure 3: Typing rules for μ -RA

$$\begin{aligned}
d(\varphi_1 \cup \varphi_2, X) &= d(\varphi_1, X) \cup d(\varphi_2, X) \\
d(\varphi_1 \triangleright \varphi_2, X) &= d(\varphi_1, X) \\
d(\varphi_1 \bowtie \varphi_2, X) &= d(\varphi_1, X) \cup d(\varphi_2, X) \\
d(\rho_a^b(\varphi), X) &= \{p \circ (b \rightarrow a, a \rightarrow \perp) \mid p \in d(\varphi, X)\} \\
d(\tilde{\pi}_a(\varphi), X) &= \{p \circ (a \rightarrow \perp) \mid p \in d(\varphi, X)\} \\
d(\sigma_{\mathfrak{f}}(\varphi), X) &= d(\varphi, X) \\
d(\mu(Y = \varphi), X) &= \emptyset \\
d(X, X) &= \{()\} \text{ (a singleton identity)} \\
d(R, X) &= \emptyset \\
d(|c \rightarrow v|, X) &= \emptyset
\end{aligned}$$

Where $\circ$ represents the composition and $(a_1 \rightarrow b_1, \dots, a_n \rightarrow b_n)$ represents the function that maps each a_i to its b_i and every other column name to itself. Note that this definition manipulates functions with an infinite domain but the domain where they do not coincide with the identity is finite and they are thus computable.

EXAMPLE 1 FOLLOWUP. In our previous example, X appears only once and thus there is only one derivation that maps $src \rightarrow \perp, m \rightarrow \perp$ and everything else to itself. In particular trg is mapped to itself.

LEMMA 2. Let w be a mapping and φ a term linear, positive and non mutually recursive in X . For all $m \in \llbracket \varphi \rrbracket_{V[X/\{w\}]}$ either $m \in \llbracket \varphi \rrbracket_{V[X/\emptyset]}$ or there exists $p \in d(\varphi, X)$ such that for all $c \in \text{dom}(w)$:

$$(p(c) = \perp) \vee (p(c) \notin \text{dom}(w)) \vee (m(c) = w(p(c)))$$

DEFINITION 10. Given a term φ linear and positive in a variable X , we define the stabilizer of X in φ as the following set of column names: $\text{stab}(\varphi, X) = \{c \in \mathbb{C} \mid \forall p \in d(\varphi, X) p(c) = c\}$

LEMMA 3. Given a fixpoint term $\mu(X = \varphi) \in \mathcal{F}[\Gamma]$ of type t and a mapping w of type t , $w \in \llbracket \mu(X = \varphi) \rrbracket_V$ if and only if there exists a lineage for w , that is, a finite sequence $w_0, \dots, w_n$ such that: $w_0 \in \llbracket \varphi \rrbracket_{V[X/\emptyset]}$, $w_{i+1} \in \llbracket \varphi \rrbracket_{V[X/\{w_i\}]}$, and $w_n = w$.

Furthermore, for all lineages $w_0, \dots, w_n$ and all $c \in t \cap \text{stab}(\varphi, X)$, we have for all i , $w_0(c) = w_i(c)$.

4.2.2 Application to rewrite rules RW1 and RW2.

THEOREM 1 (PUSHING FILTERS). Let $\mu(X = \kappa \cup \psi)$ be a decomposed fixpoint term, V an environment and $\mathfrak{f}$ a filter condition with $FC(\mathfrak{f}) \subseteq \text{stab}(\psi, X)$. Then we have $\llbracket \sigma_{\mathfrak{f}}(\mu(X = \kappa \cup \psi)) \rrbracket_V = \llbracket \mu(X = \sigma_{\mathfrak{f}}(\kappa) \cup \psi) \rrbracket_V$.

PROOF SKETCH. This is a consequence of lemma 3: we can filter the lineage on w or on w_0 but they have equal values on $FC(\mathfrak{f})$ and by definition of FC , $\text{eval}(\mathfrak{f}, w_0) = \text{eval}(\mathfrak{f}, w)$. $\square$

EXAMPLE 1 FOLLOWUP. In our previous example, trg was in the stabilizer and src was not. Indeed the term $\mu(X = R \cup \tilde{\pi}_m(\rho_{\text{trg}}^m(R) \bowtie \rho_{\text{src}}^m(X)))$ computes the pairs (src, trg) in the transitive closure of R by taking a pair (src, trg) already in the closure then finding a link (src', src) in R and finally adding (src', trg) to the set of solutions.

On this term, pushing a filter on trg is possible but not on src : a solution (src, trg) might not pass the filter but be useful to discover a solution (src', trg) passing the filter.

THEOREM 2 (PUSHING ANTI-JOINS). Let $\mu(X = \kappa \cup \psi)$ be a decomposed fixpoint term, V an environment and ξ a term of type $t \subseteq \text{stab}(\psi, X)$ (we suppose that X is not a free variable of ξ). Then we have $\llbracket \mu(X = \kappa \cup \psi) \triangleright \xi \rrbracket_V = \llbracket \mu(X = (\kappa \triangleright \xi) \cup \psi) \rrbracket_V$.

PROOF SKETCH. The anti join will act in a very similar way to a filter since ξ is constant in X . Lineages $w_0 \dots w_n$ will preserve the property to be compatible with one of the elements of $\llbracket \xi \rrbracket_V$. $\square$

4.3 Addable columns to a fixpoint

Let us assume a fixpoint $\varphi = \mu(X = \kappa \cup \psi)$ such that $\Gamma \vdash \varphi : t$. t is thus the type of κ . Suppose we change the constant part κ in such a way that it has now type $t' \neq t$. We want to ensure that the columns that t' adds or removes with respect to t play no role in the recursive computation ψ . For this, we introduce the predicate $\text{add}(\psi, X, c)$ which checks syntactically that ψ does not depend on X having a column named c .

4.3.1 Logical framework.

DEFINITION 11. We say that a column $c \in \mathbb{C}$ can be added to or removed from a term $\psi \in \mathcal{F}[\Gamma]$ recursive in X when $\text{add}(\psi, X, c) = \top$ holds, with add defined as:

$$\begin{aligned}
\text{add}(\varphi_1 \cup \varphi_2, X, c) &= \text{add}(\varphi_1, X, c) \wedge \text{add}(\varphi_2, X, c) \\
\text{add}(\varphi_1 \bowtie \varphi_2, X, c) &= \text{add}(\varphi_1, X, c) \wedge \text{add}(\varphi_2, X, c) \\
\text{add}(\varphi_1 \triangleright \varphi_2, X, c) &= \text{add}(\varphi_1, X, c) \wedge \text{add}(\varphi_2, X, c) \\
\text{add}(\rho_a^b(\varphi), X, c) &= \text{add}(\varphi, X, c) \wedge c \notin \{a, b\} \\
\text{add}(\tilde{\pi}_a(\varphi), X, c) &= \text{add}(\varphi, X, c) \text{ when } c \neq a \\
\text{add}(\tilde{\pi}_c(\varphi), X, c) &= X \notin \text{free}(\varphi) \\
\text{add}(\sigma_{\bar{f}}(\varphi), X, c) &= \text{add}(\varphi, X, c) \wedge c \notin \text{FC}(\bar{f}) \\
\text{add}(\mu(Y = \varphi), X, c) &= \text{add}(\varphi, X, c) \\
\text{add}(R, X, c) &= c \notin \Gamma(R) \text{ when } X \neq R \\
\text{add}(X, X, c) &= \top \\
\text{add}(|c' \rightarrow v|, X, c) &= c \neq c'
\end{aligned}$$

LEMMA 4. Let $\mu(X = \kappa \cup \psi) \in \mathcal{F}[\Gamma]$ be a decomposed fixpoint of type t , let $c \in (\mathcal{C} \setminus t)$ that can be added to ψ , and w a mapping of type t . We note $w(v) = w \cup \{c \rightarrow v\}$.

If $\forall R \in \mathcal{R}, c \notin \Gamma(R)$, then we have:

- $c \in \text{stab}(\psi, X)$
- $\Gamma \cup \{X \rightarrow t \cup \{c\}\} \vdash \psi : t \cup \{c\}$
- $\llbracket \psi \bowtie |c \rightarrow v| \rrbracket_{V[X/\{w\}]} = \llbracket \psi \rrbracket_{V[X/\{w(v)\}]}$

PROOF SKETCH. The first point can be proved inductively by definition of *stab* and *add*. The second point is a consequence of the first. The third point can be proved by induction on the size of ψ . $\square$

4.3.2 Application to rewrite rules RW3, RW4 and RW5.

THEOREM 3 (PUSHING JOINS). Let $\mu(X = \kappa \cup \psi) \in \mathcal{F}[\Gamma]$ be a decomposed fixpoint of type t_κ and $\varphi \in \mathcal{F}[\Gamma]$ (with $X \notin \text{free}(\varphi)$) a term of type t_φ such that:

- (1) $t_\varphi \subseteq \text{stab}(\psi, X)$
- (2) $\forall c \in t_\varphi \setminus t_\kappa \text{ add}(\psi, X, c)$

Then we have $\Gamma \vdash \mu(X = \kappa \bowtie \varphi \cup \psi) : t_\varphi \cup t_\kappa$ with for all V compatible with Γ :

$$\llbracket \varphi \bowtie \mu(X = \kappa \cup \psi) \rrbracket_V = \llbracket \mu(X = \kappa \bowtie \varphi \cup \psi) \rrbracket_V$$

PROOF SKETCH. First we prove that $\psi \in \mathcal{F}[\Gamma \cup \{X \rightarrow t_\kappa \cup t_\varphi\}]$ by iterating Lemma 4.1. Then for each lineage $w_0, \dots, w_n$ of $\llbracket \mu(X = \kappa \cup \psi) \rrbracket_V$ and each v compatible with w_0 we can build a lineage $(w_0 + v), \dots, (w_n + v)$ (by iteration on Lemma 4), which proves $w + v \in \llbracket \mu(X = \varphi \bowtie \kappa \cup \psi) \rrbracket_V$.

The reverse direction is proved similarly. $\square$

THEOREM 4 (MERGING FIXPOINTS). Given two decomposed fixpoints $\mu(X = \kappa_1 \cup \psi_1)$ and $\mu(X = \kappa_2 \cup \psi_2)$ of types t_1 and t_2 such that:

- (1) $t_1 \cap t_2 \subseteq \text{stab}(\psi_2, X, C_2) \cap \text{stab}(\psi_1, X, C_1)$
- (2) $\forall c \in t_1 \setminus t_2 \text{ add}(\psi_2, X, c)$
- (3) $\forall c \in t_2 \setminus t_1 \text{ add}(\psi_1, X, c)$

then we have: $\llbracket \mu(X = \kappa_1 \cup \psi_1) \bowtie \mu(X = \kappa_2 \cup \psi_2) \rrbracket_V = \llbracket \mu(X = \kappa_1 \bowtie \kappa_2 \cup \psi_1 \cup \psi_2) \rrbracket_V$.

PROOF SKETCH. The forward direction is easy: given two lineages $w_0^1, \dots, w_n^1$ and $w_0^2, \dots, w_m^2$ (for

both $\llbracket \mu(X = \kappa_i \cup \psi_i) \rrbracket_V$) we can build a lineage $(w_0^1 + w_0^2) \dots (w_0^1 + w_m^2) \dots (w_n^1 + w_m^2)$.

The converse direction is more difficult but we can de-interlace the lineages and create two lineages, one for each $\llbracket \mu(X = \kappa_i \cup \psi_i) \rrbracket_V$. $\square$

EXAMPLE 3. If we want to compute $R_1^+(x, y) \wedge R_2^+(y, z)$, the naive translation would compute both R_1^+ and R_2^+ . But our approach also considers the plan where we start from x, y, z such that $R_2(y, z) \wedge R_1(x, y)$ and then will discover new x or new z by a fixpoint: $\mu(X = R_1 \bowtie R_2 \cup \psi)$ with $\psi = \tilde{\pi}_c(\rho_x^c(X) \bowtie \rho_y^c(R_1)) \cup \tilde{\pi}_c(\rho_z^c(X) \bowtie \rho_y^c(R_2))$.

THEOREM 5 (PUSHING ANTIPROJECTIONS). Let $\mu(X = \kappa \cup \psi) \in \mathcal{F}[\Gamma]$ be a decomposed fixpoint of type t_κ . Let $b \in \mathcal{C}$ be such that $\text{add}(\psi, X, b)$. Then:

$$\llbracket \tilde{\pi}_b(\mu(X = \kappa \cup \psi)) \rrbracket_V = \llbracket \mu(X = \tilde{\pi}_b(\kappa) \cup \psi) \rrbracket_V$$

PROOF SKETCH. This property can be proved via lineages similarly to the proofs of the other theorems. $\square$

EXAMPLE 1 FOLLOWUP. In our running example, *trg* is an addable column. This means that, if we are interested in $\tilde{\pi}_{trg}(\mu(X = R \cup \tilde{\pi}_m(\rho_{trg}^m(R) \bowtie \rho_{src}^m(X))))$ we can push the $\tilde{\pi}_{trg}(\cdot)$ and obtain $\mu(X = \tilde{\pi}_{trg}(R \cup \tilde{\pi}_m(\rho_{trg}^m(R) \bowtie \rho_{src}^m(X))))$. Pushing $\tilde{\pi}_{trg}(\cdot)$ will reduce drastically the number of solutions. We cannot push $\tilde{\pi}_{src}(\cdot)$ as the column *src* is used.

5 GRAPH QUERY TRANSLATIONS

μ -RA can be used to model queries over directed graphs with labeled edges. We assume that the set of values $\mathcal{B}$ gathers both vertices and edge labels. The graph can then be represented as a pair $(\mathcal{V}, \mathcal{E})$ with $\mathcal{V} \subset \mathcal{B}$ denoting the set of vertices and $\mathcal{E} \subset \mathcal{V} \times \mathcal{B} \times \mathcal{V}$ denoting the set of edges. This can be modeled as a relational database with two relations V and E representing these two sets, with the schema $\Gamma = \{V \rightarrow \{\text{src}\}, E \rightarrow \{\text{src}, l, \text{trg}\}\}$ where *src*, *l*, *trg* stand respectively for *source*, *label*, *target*.

Regular path queries (RPQs) [3, 23, 27, 32, 52] provide a basic construct used in graph query languages. An RPQ makes it possible to express a path connecting graph nodes by the means of regular expressions over edge labels. We consider a set W of query variables, and a set $K \subseteq \mathcal{B}$ of vertex labels (constants). The general syntax of an RPQ is $r(x, y)$ where $x \in W \cup K$ is connected to $y \in W \cup K$ by the regular path expression r defined as follows:

$r ::=$	v	a single edge label
	r_1/r_2	concatenation
	$r_1 r_2$	alternative
	r^{-1}	reverse
	r^+	transitive closure

For example, the sample query Q_2 given in the introduction is an RPQ. The basic component for translating graph queries into μ -RA is the translation of the regular path expressions r . We translate any r into a set of μ -RA terms ϕ , representing one or more alternative translations, in such a way that the result of evaluating any of those ϕ on the graph database is the set of all mappings $\{\text{src} \rightarrow v_1, \text{trg} \rightarrow v_2\}$ such that the sequence of labels in the path from v_1 to v_2 matches r . For this purpose, we define a translation function $\langle \cdot \rangle$ that compiles any r into μ -RA, as follows:

$$\begin{aligned} \langle v \rangle &= \{\tilde{\pi}_1(\sigma_{|=v}(E))\} \\ \langle r_1/r_2 \rangle &= \{\tilde{\pi}_m(\rho_{\text{trg}}^m(\phi_1) \bowtie \rho_{\text{src}}^m(\phi_2)) \mid \phi_1 \in \langle r_1 \rangle \wedge \phi_2 \in \langle r_2 \rangle\} \\ \langle r_1|r_2 \rangle &= \{\phi_1 \cup \phi_2 \mid \phi_1 \in \langle r_1 \rangle \wedge \phi_2 \in \langle r_2 \rangle\} \\ \langle r^{-1} \rangle &= \{\rho_{\text{src}}^{\text{trg}}(\rho_{\text{trg}}^m(\phi)) \mid \phi \in \langle r \rangle\} \\ \langle r^+ \rangle &= \left\{ \mu(X = \phi \cup \tilde{\pi}_m(\rho_{\text{trg}}^m(\phi) \bowtie \rho_{\text{src}}^m(X))) \mid \phi \in \langle r \rangle \right\} \cup \\ &\quad \left\{ \mu(X = \phi \cup \tilde{\pi}_m(\rho_{\text{src}}^m(\phi) \bowtie \rho_{\text{trg}}^m(X))) \mid \phi \in \langle r \rangle \right\} \end{aligned}$$

Notice that for r^+ we have two equivalent translated terms. This is because we can choose to rename src to m either in r or in X (and correspondingly trg to m in the other), depending on the direction we want to follow when recursively navigating the graph. We want to keep track of both translations, as this has impact on the plan space generation¹. This is the reason why $\langle r \rangle$ returns a set of terms and not a single term. This translation of regular path expressions constitutes the main component used for translating graph query languages such as UCRPQs.

A CRPQ is of the form $H \leftarrow C$, where the query head H is a non-empty set of vertex variables to be extracted by the query, and C is a conjunction of RPQs that describes how those vertex variables are connected to other vertex variables or to constants. More formally:

- H is of the form $(z_1, \dots, z_m)$ with arity $m > 0$ (we do not consider boolean queries)
- C is a conjunction of the form $r_1(x_1, y_1), \dots, r_n(x_n, y_n)$ where:
 - $x_1, y_1, \dots, x_n, y_n \in W \cup K$
 - each r_i is an RPQ (as defined previously)
 - for each $0 < k \leq m$ we have $z_k \in \{x_1, y_1, \dots, x_n, y_n\} \setminus K$.

UCRPQs extend CRPQs with union at top level. They have the syntax $H \leftarrow C_1 \cup \dots \cup C_n$ in which each disjunct C_i is a conjunction as defined previously. We translate a UCRPQ into μ -RA as follows: $\langle H \leftarrow C_1 \cup C_2 \dots \cup C_n \rangle = \langle C_1 \rangle_H \cup \langle C_2 \rangle_H \dots \cup \langle C_n \rangle_H$ with:

¹This is because rewrite rules presented in § 4 apply differently over each initial translation, generating two different plan spaces, and we want to explore the union of the two.

$$\langle C \rangle_H = \{\phi_1 \bowtie \phi_2 \bowtie \dots \bowtie \phi_n \mid (\phi_1, \phi_2, \dots, \phi_n) \in \text{combine}(C)_H\}$$

$$\begin{aligned} \text{combine}(r_1(x_1, y_1), \dots, r_n(x_n, y_n))_H \\ = \langle r_1(x_1, y_1) \rangle_H \times \dots \times \langle r_n(x_n, y_n) \rangle_H \end{aligned}$$

$$\begin{aligned} \langle r(x, y) \rangle_H &= \left\{ \Pi \left(\theta_{\text{src}}^x \left(\theta_{\text{trg}}^y(\phi) \right) \right)_H \mid \phi \in \langle r \rangle \right\} \\ \theta_c^x(\phi) &= \begin{cases} \rho_c^x(\phi) & \text{for } x \in W \\ \sigma_{c=x}(\phi) & \text{for } x \in K \end{cases} \quad \text{in which} \end{aligned}$$

$\Pi(\phi)_H = \tilde{\pi}_{x_1}(\tilde{\pi}_{x_2} \dots (\tilde{\pi}_{x_n}(\phi)))$ where $x_1, x_2, \dots, x_n$ occur in ϕ but not in H (we keep only columns corresponding to selected variables).

6 EXPERIMENTS

6.1 Prototype

For shedding light on the practical interest of computing richer plan spaces, we experimentally compared the performance of graph query evaluation with and without our novel rewriting rules. To test these rules, we implemented a prototype executor for UCRPQs as follows. First, queries are translated into μ -RA terms. Then, we enumerate equivalent terms using a Volcano-style strategy [37]. We then pick one of the equivalent terms, translate it into SQL, and send it to PostgreSQL. Physical plan selection and evaluation are left to the PostgreSQL engine; however, depending on the μ -RA term we pick, this engine will not be able to access the same plan space.

The key point is the set of fixpoint subterms in the query. Each of those will always be computed in full into a temporary table by the engine, since it has no internal rules to move other operations in or out of the recursion block. Based on this observation, we estimate a cost for each μ -RA term and pick one of the terms with the minimal estimated cost (see below).

We have two possible translations for fixpoint terms. When a fixpoint $\mu(X = \kappa \cup \psi)$ is such that X appears exactly once in ψ , which is always the case for terms generated by translating a UCRPQ, this fixpoint can be translated into a CREATE RECURSIVE VIEW statement, which is purely declarative so that the engine can freely decide how to compute it. This allows us to test PostgreSQL on the initial translated query without restricting its plan space (this test is what we call ‘System P’ below). However, the rewriting rule for merging fixpoints generates terms where X appears twice; these terms cannot be translated into pure SQL. In that case, we resort to procedural language (PL/pgSQL) and a WHILE loop which computes a temporary table containing the result of the fixpoint term.

Cost estimation. We use a simple cost estimation based on [60], extended to deal with fixpoint terms. It uses a few parameters: number of tuples in each relation in the database, number of distinct values for each column, and histogram

values for selectivity [42]. These parameters are available from the PostgreSQL server where the data is stored. For a fixpoint term $\mu(X = \kappa \cup \psi)$, we assume that the number of new tuples generated is reduced, at each iteration, by a factor s which we compute from selectivity values and the operations in ψ . As the iteration stops when no new tuples are generated, we estimate the number of steps as $\log_s(K)$ where K is the estimated number of tuples in κ . This estimated number of steps allows us to estimate, in turn, the number of tuples in $\mu(X = \kappa \cup \psi)$ and the CPU time required.

6.2 Methodology

6.2.1 Queries. Our methodology is twofold: first, we tested a set of realistic queries evaluated over a real-world dataset; and we also used the gMark benchmark [15] to randomly generate UCRPQs with corresponding synthetic datasets.

Queries over the YAGO dataset. We use a cleaned version of the real world dataset YAGO2s [33], that we have preprocessed in order to remove duplicate triples and keep only triples with existing and valid identifiers. After preprocessing, we obtain a table of YAGO facts with 83 predicates and 62,643,951 rows (graph edges). We collected third-party queries previously considered against this popular dataset: we collected 7 RPQs from [5], 2 from [70] and 2 from [41]. Since all of them are RPQs, we supplemented them with additional CRPQs to illustrate more complex forms of recursion. Fig. 4 presents the 20 queries tested with the YAGO dataset.

?x	←	?x isMarriedTo/livesIn/IsL+/dw+ Argentina	Q ₁
?x	←	?x hasChild/livesIn/IsL+/dw+ Japan	Q ₂
?x	←	?x influences/livesIn/IsL+/dw+ Sweden	Q ₃
?x	←	?x livesIn/IsL+/dw+ United_States	Q ₄
?x	←	?x hasSuccessor/livesIn/IsL+/dw+ India	Q ₅
?x	←	?x hasPredecessor/livesIn/IsL+/dw+ Germany	Q ₆
?x	←	?x haa/livesIn/IsL+/dw+ Netherlands	Q ₇
?x	←	?x IsL+/dw+ USA	Q ₈
?x	←	?x (actedIn/-actedIn)+ Kevin_Bacon	Q ₉
?area	←	wce -typ/(IsL+/dw dw) ?area	Q ₁₀
?p	←	?p isMarriedTo/owns/IsL+ owns/IsL+ USA	Q ₁₁
?a,?b	←	?a IsL+/dw ?b	Q ₁₂
?a,?b	←	?a IsL+/dw+ ?b	Q ₁₃
?a,?b,?c	←	?a wasBornIn/IsL+ ?b, ?b isConnectedTo+ ?c	Q ₁₄
?a,?b,?c	←	?a (IsL isConnectedTo)+ ?b, ?c wasBornIn ?a	Q ₁₅
?a,?c	←	?a wasBornIn/IsL+ Japan, ?a typ/sc ?c	Q ₁₆
?a	←	?a IsL+/(isConnectedTo dw)+ Japan	Q ₁₇
?a,?c	←	?a IsL+ Japan, ?a isConnectedTo+ ?c	Q ₁₈
?a	←	?a IsL+/IsL Japan	Q ₁₉
?a	←	?a IsL+/isConnectedTo+/dw+ Japan	Q ₂₀

Figure 4: Queries for the YAGO dataset^a.

^aQ₁...Q₇ are taken from [5], Q₈, Q₉ from [70], and Q₁₀, Q₁₁ from [41].
 “isL” stands for “IsLocatedIn”, “dw” for “dealsWith”, “haa” for
 “hasAcademicAdvisor”, “USA” for “United_States”, “wce” for
 “wikicategory_Capitals_in_Europe”, “typ” for “rdf:type” and “sc” for
 “rdfs:subClassOf”.

Generated queries over synthetic datasets, using gMark. We gathered queries generated by gMark [15], which are available on the gMark open source repository [16]. We filtered them to retain only queries in which at least one recursion was present. For queries generated with an empty head, we replaced the empty head by a head containing all the variables occurring in the body (so as to test a more complex polyadic variant of the query instead of its simple boolean counterpart). This provided us with 12 recursive queries for the “UniProt” gMark scenario, that we evaluated on a gMark-generated graph instance having 76,707 edges. We also carried out similar tests with the “Shop” gMark scenario for which we report on the evaluation of 14 queries over a synthesized graph instance with 209,789 edges. Table 1 presents the sizes of the considered datasets.

Dataset Origin	Predicates	Edges	Nodes
YAGO 2.5 (cleaned) [33]	83	62,643,951	42,832,856
gMark-Shop [15]	81	209,789	135,737
gMark-Uniprot [15]	7	76,707	21,130

Table 1: Dataset Statistics.

6.2.2 Tested Systems. We compared the query evaluation performance of the newly obtained recursive query plans with state-of-the-art systems implementing previously known approaches for recursive queries, namely:

- system **P**: the popular PostgreSQL open-source relational database [56, 63] implementing SQL with recursive views.
- system **P'**: our prototype extending the PostgreSQL system with our optimizations;
- system **V**: the Virtuoso graph column store [30] which is backed by a relational database, and that implements the SPARQL 1.1 language [43] (with property paths);
- system **L**: a modern engine implementing Datalog [66];
- system **N**: the Neo4j native graph database implementing the openCypher graph query language [40, 55].

6.3 Experimental setup

Set semantics. Most systems implement both bag and set semantics, and use bag semantics by default. However, their implementation of recursion differs significantly, which causes some systems to retrieve more or less duplicates when compared to others. Therefore, we use set semantics.

Timeout. For each tested system, we set a maximum time of 30 min of computation for each query (after 30 min, we stop the computations and consider that this particular query evaluation is not feasible with the given system)².

²Experiments have been conducted on a server with 128 GB of RAM, 2 Intel Xeon E5-2630 v4 CPUs (2.20 GHz, 20 cores each) and 66 TB of 7200 RPM

Specific limitations. Some systems have inherent limitations or require specific configuration. System **N** supports recursion only around atomic patterns (e.g. `isLocatedIn+` is supported but `(actedIn/-actedIn)+` is not) so it can evaluate only a limited number of the considered queries.

*Initial comparison baseline for system **P**.* A given graph query translates into several possible μ -RA forms, even before applying our rewrite rules (see § 5). For a fair comparison with system **P**, we use the internal cost estimation of **P** to discriminate them³. Thus, for a given query, the time reported for system **P** is the elapsed time for evaluating the query translation that **P** has itself chosen among other equivalent initial translations.

Reported metrics. We report on query evaluation performance, excluding (i) time spent for data preparation (e.g. for loading or computing indexes) and (ii) for query optimization (e.g. for generating the plan space – a task which can benefit from extensive research and various techniques found in the literature [38, 39, 47, 62], and which is beyond the scope of this paper). We thus mainly concentrate on query evaluation times. We also report on the number of query answers returned by each system using set semantics.

6.4 Results

6.4.1 Queries over the YAGO dataset. Fig. 5 shows the time spent with each system for evaluating each query of Fig. 4. The time scale is logarithmic. Whenever no time is reported for a given query and system, this means that either the system crashed (or answered wrongly) within the first 30 minutes of computation for that query, or the computations were stopped after 30 minutes. In both cases, the evaluation of the query is considered unfeasible with that system.

Coverage. First, we observe significant discrepancies between the number of queries that each system has been able to answer. Systems **N** and **V** answered only 4 queries out of 20. In comparison, system **P** answered many more queries: all but one; and System **P'** answered all of them. Table 2 summarizes the number of feasible queries for each system. Fig. 6 presents the number of retrieved results (in logarithmic scale) for each query and each system. All the systems agree on the number of query results (using set semantics), except **V** which did not retrieve the correct results in some cases (typically terminating within the allowed timeframe but incorrectly returning no result). In those cases, we consider that it could not answer the query, and no time is reported.

hard disk drives (SAS, RAID 5), running Ubuntu 16.04 LTS, Docker 18.09.7, and the latest public Docker images for each tested system.

³Specifically, for each initial translation, we get the ‘total cost’ value returned by a call to the “EXPLAIN” statement of system **P**. We then retain the translation of minimum estimated cost.

	N	V	L	P	P'
Feasible queries	4	4	17	19	20
Unfeasible queries	16	16	3	1	0

Table 2: YAGO queries treated by each system.

Performance for feasible queries. We observe that **P'** always performed more efficiently than **P**, in several cases by an order of magnitude. This suggests that the new plans computed by our approach sometimes represent much more efficient alternatives. In the case of query 9, the new plan selected even makes query answering possible whereas it is unfeasible with other systems. Table 3 summarizes the gains brought by **P'**, in terms of feasibility and performance, in comparison to other systems.

	P	N	V	L
Q_1	131	34.6	∞	88.6
Q_2	211	∞	∞	143
Q_3	198	44.8	∞	132
Q_4	2.1	∞	∞	1.4
Q_5	198	∞	∞	136
Q_6	171	∞	∞	115
Q_7	346	∞	∞	367
Q_8	1.0	∞	∞	0.7
Q_9	∞	∞	∞	∞
Q_{10}	2.2	∞	∞	∞

	P	N	V	L
Q_{11}	4.5	∞	0.04	∞
Q_{12}	2.3	∞	0.3	2.5
Q_{13}	1.4	∞	∞	1.0
Q_{14}	2.4	∞	0.02	2.2
Q_{15}	2.2	∞	∞	2.5
Q_{16}	63.1	10.6	∞	312
Q_{17}	5.8	∞	∞	7.1
Q_{18}	38.4	∞	∞	39.9
Q_{19}	573	146	0.4	664
Q_{20}	10.8	∞	∞	14.3

Table 3: Speedup with **P' for YAGO queries^a.**

^aA speedup $x > 1$ means query evaluation is x times faster, $x < 1$ means slower, $x = 1$ means no speedup, and ∞ denotes cases where a formerly unfeasible query becomes feasible.

6.4.2 Generated queries over synthetic datasets. Fig. 7 and 8 show the times spent in evaluating recursive queries of the “UniProt” and the “Shop” gMark scenarios [15], respectively. No results are reported for system **N** because it supports none of the randomly generated queries (each one contains at least one form of recursion such as $(a/b)^+$ which is not supported by **N**). Results show that in the majority of cases, **P'** outperforms the other systems, either in terms of feasibility or in terms of performance. A few cases clearly illustrate the interest of the rich plan space explored by system **P'**. For instance with UniProt queries 3, 5, 6, 8, 12, and with Shop queries 2, 5, 6, 8, 9, 10, 13, system **P'** performs much more efficiently thanks to the selected plan which was not present in the plan space of system **P**. Some cases also illustrate that there is still room for improvement of the approach, in particular for the cost estimation function in charge of picking the best estimated plan in the search space. This is the case for UniProt query 1 and Shop query 4 for instance, where the plan picked by system **P'** reveals less efficient than

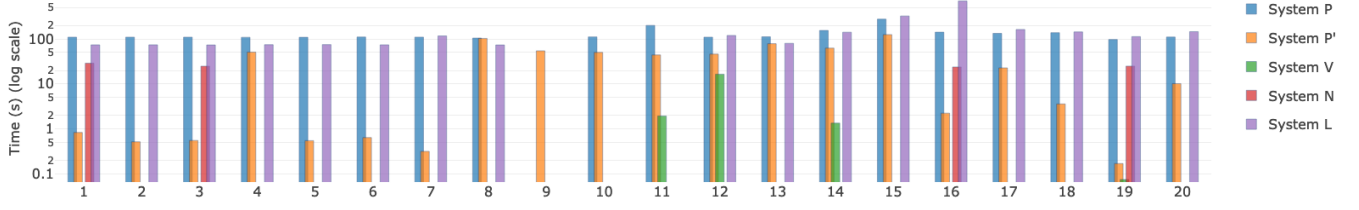


Figure 5: Elapsed time for evaluating queries of Fig. 4 over the YAGO real-world dataset.

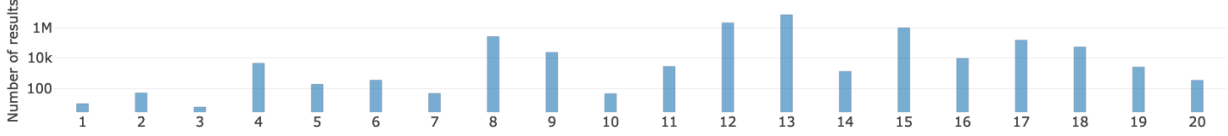


Figure 6: Number of results retrieved (with set semantics) by each query of Fig. 4 on the YAGO dataset.

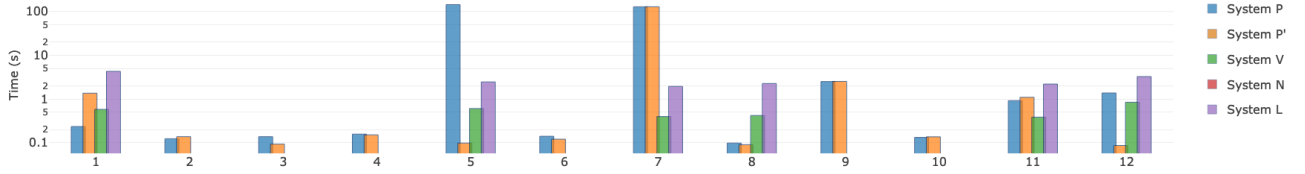


Figure 7: Time spent for evaluating recursive queries of the synthetic “UniProt” gMark scenario.

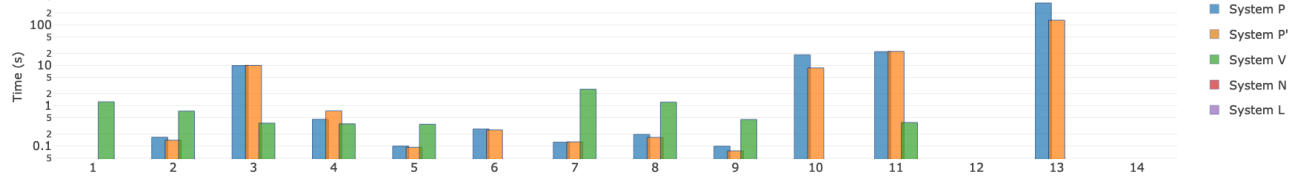


Figure 8: Time spent for evaluating recursive queries of the synthetic “Shop” gMark scenario.

the plan of system **P**. The term-picking function should thus ideally favor the initial plan, also present in the search space.

Overall, experiments suggest that the new plans computed by our approach can offer significant gains in computation time when evaluating recursive queries over graphs. They also suggest that some of the newly computed plans can allow to realize certain query evaluations which used to be unfeasible. This illustrates the benefits of the exploration of richer plan spaces made possible by μ -RA.

7 CONCLUSION

We propose a variation of the classical relational algebra extended with a fixpoint operator, which is useful for capturing recursive terms and for facilitating their transformations. We propose new rewriting rules for recursive terms. These new rules are compatible and compositional with existing

rules for optimizing the core of the relational algebra. The extended set of optimization rules makes it possible to compute new query execution plans beyond reach with previous approaches. Our approach can be used within any mainstream database management system that implements SQL with recursion, either by adding the new rules inside the query optimizer, or as a preprocessing stage not requiring to modify the system’s internals. Experiments with a prototype implementing such a preprocessing phase for the PostgreSQL system suggest that the new query plans can be useful for evaluating much more efficiently recursive queries over graphs. These works open many interesting perspectives for future work, including more precise cardinality estimations for term selection, and the study of further algebraic extensions with aggregation and user-defined functions in particular.

REFERENCES

- [1] Andrejs Abele, John P. McCrae, Paul Buitelaar, Anja Jentzsch, and Richard Cyganiak. 2017. Linking Open Data cloud diagram. <http://lod-cloud.net/>
- [2] Serge Abiteboul, Richard Hull, and Victor Vianu (Eds.). 1995. *Foundations of Databases: The Logical Level* (1st ed.). Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA.
- [3] S. Abiteboul, D. Quass, J. McHugh, J. Widom, and J. Wiener. 1996. The Lorel Query Language for Semistructured Data. *Journal on Digital Libraries* 1, 1 (1996). <http://ilpubs.stanford.edu:8090/162/>
- [4] Serge Abiteboul and Victor Vianu. 1991. Datalog Extensions for Database Queries and Updates. *J. Comput. Syst. Sci.* 43, 1 (Aug. 1991), 62–124. [https://doi.org/10.1016/0022-0000\(91\)90032-Z](https://doi.org/10.1016/0022-0000(91)90032-Z)
- [5] Zahid Abul-Basher, Nikolay Yakovets, Parke Godfrey, Shadi Ghajar-Khosravi, and Mark H. Chignell. 2017. TASWEET: Optimizing Disjunctive Path Queries in Graph Databases. In *Proceedings of the 20th International Conference on Extending Database Technology, EDBT 2017, Venice, Italy, March 21–24, 2017*. OpenProceedings.org, 470–473. <https://doi.org/10.5441/002/edbt.2017.47>
- [6] Foto Afrati, Manolis Gergatsoulis, and Francesca Toni. 2003. Linearisability on datalog programs. *Theoretical Computer Science* 308, 1 (2003), 199 – 226. [https://doi.org/10.1016/S0304-3975\(02\)00730-2](https://doi.org/10.1016/S0304-3975(02)00730-2)
- [7] R. Agrawal. 1988. Alpha: an extension of relational algebra to express a class of recursive queries. *IEEE Transactions on Software Engineering* 14, 7 (July 1988), 879–885. <https://doi.org/10.1109/32.42731>
- [8] Alfred V. Aho and Jeffrey D. Ullman. 1979. Universality of Data Retrieval Languages. In *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (San Antonio, Texas) (POPL ’79). ACM, New York, NY, USA, 110–119. <https://doi.org/10.1145/567752.567763>
- [9] Alexander Alexandrov, Georgi Krastev, and Volker Markl. 2019. Representations and Optimizations for Embedded Parallel Dataflow Languages. *ACM Trans. Database Syst.* 44, 1, Article 4 (Jan. 2019), 44 pages. <https://doi.org/10.1145/3281629>
- [10] Faisal Alkhateeb and Jérôme Euzenat. 2014. Constrained regular expressions for answering RDF-path queries modulo RDFS. *IJWIS* 10, 1 (2014), 24–50. <https://doi.org/10.1108/IJWIS-05-2013-0013>
- [11] Renzo Angles, Marcelo Arenas, Pablo Barcelo, Peter Boncz, George Fletcher, Claudio Gutierrez, Tobias Lindaaaker, Marcus Paradies, Stefan Plantikow, Juan Sequeda, Oskar van Rest, and Hannes Voigt. 2018. G-CORE: A Core for Future Graph Query Languages. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) (SIGMOD ’18). 1421–1432. <https://doi.org/10.1145/3183713.3190654>
- [12] Anonymous. 2019. Full proofs for SIGMOD2020 submission 48. <https://github.com/asigmod/SIGMOD2020submission48>
- [13] Molham Aref, Balder ten Cate, Todd J. Green, Benny Kimelfeld, Dan Olteanu, Emir Pasalic, Todd L. Veldhuizen, and Geoffrey Washburn. 2015. Design and Implementation of the LogicBlox System. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* (Melbourne, Victoria, Australia) (SIGMOD ’15). ACM, New York, NY, USA, 1371–1382. <https://doi.org/10.1145/2723372.2742796>
- [14] Marcelo Arenas, Sebastián Conca, and Jorge Pérez. 2012. Counting Beyond a Yottabyte, or How SPARQL 1.1 Property Paths Will Prevent Adoption of the Standard. In *Proceedings of the 21st International Conference on World Wide Web (WWW ’12)*. ACM, New York, NY, USA, 629–638. <https://doi.org/10.1145/2187836.2187922>
- [15] Guillaume Bagan, Angela Bonifati, Radu Ciucanu, George H. L. Fletcher, Aurélien Lemay, and Nicky Advokaat. 2017. gMark: Schema-Driven Generation of Graphs and Queries. *IEEE Trans. Knowl. Data Eng.* 29, 4 (2017), 856–869. <https://doi.org/10.1109/TKDE.2016.2633993>
- [16] Guillaume Bagan, Angela Bonifati, Radu Ciucanu, George H. L. Fletcher, Aurélien Lemay, and Nicky Advokaat. 2019. gMark: a domain and query language independent framework. <https://github.com/graphMark/gmark>
- [17] Francois Bancilhon, David Maier, Yehoshua Sagiv, and Jeffrey D Ullman. 1986. Magic Sets and Other Strange Ways to Implement Logic Programs (Extended Abstract). In *Proceedings of the Fifth ACM SIGACT-SIGMOD Symposium on Principles of Database Systems* (Cambridge, Massachusetts, USA) (PODS ’86). ACM, New York, NY, USA, 1–15. <https://doi.org/10.1145/6012.15399>
- [18] Francois Bancilhon and Raghu Ramakrishnan. 1988. An amateur’s introduction to recursive query processing strategies. In *Readings in Artificial Intelligence and Databases*. Elsevier, 376–430.
- [19] Pablo Barcelo, Diego Figueira, and Leonid Libkin. 2012. Graph Logics with Rational Relations and the Generalized Intersection Problem. In *Proceedings of the 2012 27th Annual IEEE/ACM Symposium on Logic in Computer Science* (New Orleans, Louisiana) (LICS ’12). IEEE Computer Society, Washington, DC, USA, 115–124. <https://doi.org/10.1109/LICS.2012.23>
- [20] Pablo Barceló, Leonid Libkin, Anthony W. Lin, and Peter T. Wood. 2012. Expressive Languages for Path Queries over Graph-Structured Data. *ACM Trans. Database Syst.* 37, 4, Article 31 (Dec. 2012), 46 pages. <https://doi.org/10.1145/2389241.2389250>
- [21] Angela Bonifati, George Fletcher, Hannes Voigt, and Nikolay Yakovets. 2018. *Querying Graphs*. Morgan and Claypool publishers.
- [22] Angela Bonifati, Wim Martens, and Thomas Timm. 2017. An Analytical Study of Large SPARQL Query Logs. *PVLDB* 11, 2 (2017), 149–161. <https://doi.org/10.14778/3149193.3149196>
- [23] Peter Buneman, Susan Davidson, Gerd Hillebrand, and Dan Suciu. 1996. A Query Language and Optimization Techniques for Unstructured Data. *SIGMOD Rec.* 25, 2 (June 1996), 505–516. <https://doi.org/10.1145/235968.233368>
- [24] Ashok K. Chandra. 1981. Programming Primitives for Database Languages. In *Proceedings of the 8th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Williamsburg, Virginia) (POPL ’81). ACM, New York, NY, USA, 50–62. <https://doi.org/10.1145/567532.567537>
- [25] Edgar F Codd. 1970. A relational model of data for large shared data banks. *Commun. ACM* 13, 6 (1970), 377–387.
- [26] Mariano P. Consens and Alberto O. Mendelzon. 1990. GraphLog: A Visual Formalism for Real Life Recursion. In *Proceedings of the Ninth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems* (Nashville, Tennessee, USA) (PODS ’90). ACM, New York, NY, USA, 404–416. <https://doi.org/10.1145/298514.298591>
- [27] Isabel F. Cruz, Alberto O. Mendelzon, and Peter T. Wood. 1987. A Graphical Query Language Supporting Recursion. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data* (San Francisco, California, USA) (SIGMOD ’87). ACM, New York, NY, USA, 323–330. <https://doi.org/10.1145/38713.38749>
- [28] DLV Systems. 2012. The DLV deductive database system. <http://www.dlvsystem.com/dlv/> (retrieved in october 2019).
- [29] Oliver M Duschka, Michael R Genesereth, and Alon Y Levy. 2000. Recursive query plans for data integration. *The Journal of Logic Programming* 43, 1 (2000), 49–73.
- [30] Orri Erling. 2012. Virtuoso, a Hybrid RDBMS/Graph Column Store. *IEEE Data Eng. Bull.* 35, 1 (2012), 3–8. <http://sites.computer.org/debull/A12mar/vicol.pdf>
- [31] M. Fernandez and D. Suciu. 1998. Optimizing regular path expressions using graph schemas. In *Proceedings 14th International Conference on Data Engineering*. 14–23. <https://doi.org/10.1109/ICDE.1998.655753>
- [32] Daniela Florescu, Alon Levy, and Alberto Mendelzon. 1998. Database Techniques for the World-Wide Web: A Survey. *SIGMOD Rec.* 27, 3 (Sept. 1998), 59–74. <https://doi.org/10.1145/290593.290605>

- [33] Max Planck Institute for Informatics and Telecom ParisTech University. 2019. YAGO: A high-quality knowledge base. <https://www.mpi-inf.mpg.de/yago-naga/yago/>
- [34] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) (SIGMOD '18). 1433–1445. <https://doi.org/10.1145/3183713.3190657>
- [35] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An Evolving Query Language for Property Graphs. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) (SIGMOD '18). ACM, New York, NY, USA, 1433–1445. <https://doi.org/10.1145/3183713.3190657>
- [36] Georges Gardarin and Christophe de Maindreville. 1986. Evaluation of Database Recursive Logic Programs As Recurrent Function Series. In *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data* (SIGMOD '86). ACM, New York, NY, USA, 177–186. <https://doi.org/10.1145/16894.16872>
- [37] Goetz Graefe. 1994. Volcano - An Extensible and Parallel Query Evaluation System. *IEEE Trans. Knowl. Data Eng.* 6, 1 (1994), 120–135. <https://doi.org/10.1109/69.273032>
- [38] Goetz Graefe. 1995. The Cascades Framework for Query Optimization. *Data Engineering Bulletin* 18 (1995).
- [39] Goetz Graefe and William J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. In *Proceedings of the Ninth International Conference on Data Engineering*. IEEE Computer Society, Washington, DC, USA, 209–218. <http://dl.acm.org/citation.cfm?id=645478.757691>
- [40] Alastair Green, Martin Junghanns, Max Kießling, Tobias Lindaaker, Stefan Plantikow, and Petra Selmer. 2018. openCypher: New Directions in Property Graph Querying. In *Proceedings of the 21th International Conference on Extending Database Technology, EDBT 2018, Vienna, Austria, March 26-29, 2018*. 520–523. <https://doi.org/10.5441/002/edbt.2018.62>
- [41] Andrey Gubichev, Srikanta J. Bedathur, and Stephan Seufert. 2013. Sparqling Kleene: Fast Property Paths in RDF-3X. In *First International Workshop on Graph Data Management Experiences and Systems* (New York, New York) (GRADES '13). ACM, New York, NY, USA, Article 14, 7 pages. <https://doi.org/10.1145/2484425.2484443>
- [42] Peter J. Haas, Jeffrey F. Naughton, S. Seshadri, and Arun N. Swami. 1996. Selectivity and Cost Estimation for Joins Based on Random Sampling. *J. Comput. Syst. Sci.* 52, 3 (June 1996), 550–569. <https://doi.org/10.1006/jcss.1996.0041>
- [43] Steve Harris and Andy Seaborne. 2013. SPARQL 1.1 Query Language, W3C Recommendation. <https://www.w3.org/TR/sparql11-query/>
- [44] Olaf Hartig and Giuseppe Pirrò. 2017. SPARQL with property paths on the Web. *Semantic Web* 8, 6 (2017), 773–795. <https://doi.org/10.3233/SW-160237>
- [45] Maurice AW Houtsma and Peter MG Apers. 1992. Algebraic optimization of recursive queries. *Data & Knowledge Engineering* 7, 4 (1992), 299–325.
- [46] Michael Kifer and Eliezer L. Lozinskii. 1990. On Compile-time Query Optimization in Deductive Databases by Means of Static Filtering. *ACM Trans. Database Syst.* 15, 3 (Sept. 1990), 385–426. <https://doi.org/10.1145/88636.87121>
- [47] Donald Kossmann and Konrad Stocker. 2000. Iterative Dynamic Programming: A New Class of Query Optimization Algorithms. *ACM Trans. Database Syst.* 25, 1 (March 2000), 43–82. <https://doi.org/10.1145/352958.352982>
- [48] Dexter Kozen. 1983. Results on the Propositional mu-Calculus. *Theor. Comput. Sci.* 27 (1983), 333–354. [https://doi.org/10.1016/0304-3975\(82\)90125-6](https://doi.org/10.1016/0304-3975(82)90125-6)
- [49] Nicola Leone, Gerald Pfeifer, Wolfgang Faber, Thomas Eiter, Georg Gottlob, Simona Perri, and Francesco Scarcello. 2006. The DLV System for Knowledge Representation and Reasoning. *ACM Trans. Comput. Logic* 7, 3 (July 2006), 499–562. <https://doi.org/10.1145/1149114.1149117>
- [50] Leonid Libkin, Wim Martens, and Domagoj Vrgoč. 2016. Querying Graphs with Data. *J. ACM* 63, 2, Article 14 (March 2016), 53 pages. <https://doi.org/10.1145/2850413>
- [51] Erik Meijer and Gavin M. Bierman. 2011. A co-relational model of data for large shared data banks. *Commun. ACM* 54, 4 (2011), 49–58. <https://doi.org/10.1145/1924421.1924436>
- [52] Alberto O. Mendelzon and Peter T. Wood. 1995. Finding Regular Simple Paths in Graph Databases. *SIAM J. Comput.* 24, 6 (Dec. 1995), 1235–1258. <https://doi.org/10.1137/S009753979122370X>
- [53] J. F. Naughton, R. Ramakrishnan, Y. Sagiv, and J. D. Ullman. 1989. Efficient Evaluation of Right-, Left-, and Multi-linear Rules. In *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data* (Portland, Oregon, USA) (SIGMOD '89). ACM, New York, NY, USA, 235–242. <https://doi.org/10.1145/67544.66948>
- [54] Van-Quyet Nguyen and Kyungbaek Kim. 2017. Estimating the Evaluation Cost of Regular Path Queries on Large Graphs. In *Proceedings of the Eighth International Symposium on Information and Communication Technology* (Nha Trang City, Viet Nam) (SoICT 2017). ACM, New York, NY, USA, 92–99. <https://doi.org/10.1145/3155133.3155160>
- [55] Online. 2019. <http://www.opencypher.org>.
- [56] Online. 2019. The PostgreSQL system: <http://www.postgresql.org>.
- [57] John D. Ramsdell. 2004. Datalog version 2.2, a lightweight deductive database system. <http://www.ccs.neu.edu/home/ramsdell/tools/datalog/datalog.html> (retrieved in october 2019).
- [58] Domenico Saccà and Carlo Zaniolo. 1986. On the Implementation of a Simple Class of Logic Queries for Databases. In *Proceedings of the Fifth ACM SIGACT-SIGMOD Symposium on Principles of Database Systems* (Cambridge, Massachusetts, USA) (PODS '86). ACM, New York, NY, USA, 16–23. <https://doi.org/10.1145/6012.6013>
- [59] Michael Schmidt, Michael Meier, and Georg Lausen. 2010. Foundations of SPARQL Query Optimization. In *Proceedings of the 13th International Conference on Database Theory (ICDT '10)*. ACM, New York, NY, USA, 4–33. <https://doi.org/10.1145/1804669.1804675>
- [60] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, Boston, Massachusetts, USA, May 30 - June 1*. 23–34. <https://doi.org/10.1145/582095.582099>
- [61] Alexander Shkapsky, Mohan Yang, Matteo Interlandi, Hsuan Chiu, Tyson Condie, and Carlo Zaniolo. 2016. Big Data Analytics with Datalog Queries on Spark. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD '16)*. 1135–1149. <https://doi.org/10.1145/2882903.2915229>
- [62] Michael Stillger and Myra Spiliopoulou. 1996. Genetic Programming in Database Query Optimization. In *Proceedings of the 1st Annual Conference on Genetic Programming* (Stanford, California). MIT Press, Cambridge, MA, USA, 388–393. <http://dl.acm.org/citation.cfm?id=1595536.1595591>
- [63] Michael Stonebraker, Lawrence A. Rowe, and Michael Hirohama. 2019. The implementation of POSTGRES. In *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*. 519–559. <https://doi.org/10.1145/3226595.3226639>
- [64] Fabian M. Suchanek, Gjergji Kasneci, and Gerhard Weikum. 2007. Yago: a core of semantic knowledge. In *Proceedings of the 16th International Conference on World Wide Web, WWW 2007, Banff, Alberta, Canada,*

- May 8-12, 2007. 697–706. <https://doi.org/10.1145/1242572.1242667>
- [65] K Tuncay Tekle and Yanhong A Liu. 2011. More efficient datalog queries: subsumptive tabling beats magic sets. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of data*. ACM, 661–672.
- [66] Jeffrey D. Ullman. 1988. *Principles of Database and Knowledge-base Systems, Vol. I*. Computer Science Press, Inc., New York, NY, USA.
- [67] Jacopo Urbani, Criel JH Jacobs, and Markus Krötzsch. 2016. Column-Oriented Datalog Materialization for Large Knowledge Graphs.. In *AAAI*. 258–264.
- [68] Jacopo Urbani, Criel J. H. Jacobs, and Markus Krötzsch. 2016. VLog: A Column-Oriented Datalog System for Large Knowledge Graphs. In *Proceedings of the ISWC 2016 Posters & Demonstrations Track co-located with 15th International Semantic Web Conference (ISWC 2016), Kobe, Japan, October 19, 2016*. <http://ceur-ws.org/Vol-1690/paper113.pdf>
- [69] Nikolay Yakovets, P Godfrey, and J Gryz. 2013. Evaluation of SPARQL property paths via recursive SQL. 1087 (01 2013).
- [70] Nikolay Yakovets, Parke Godfrey, and Jarek Gryz. 2015. WAVEGUIDE: Evaluating SPARQL Property Path Queries.. In *EDBT*. 525–g528.
- [71] Nikolay Yakovets, Parke Godfrey, and Jarek Gryz. 2016. Query Planning for Evaluating SPARQL Property Paths. In *Proceedings of the 2016 International Conference on Management of Data (San Francisco, California, USA) (SIGMOD '16)*. ACM, New York, NY, USA, 1875–1889. <https://doi.org/10.1145/2882903.2882944>

A PROOFS FOR SECTION 3 (THE μ -EXTENDED REL. ALGEBRA)

PROPOSITION (1). If $\mu(X = \varphi)$ is linear, positive and non mutually recursive then the function $f(S) = \llbracket \varphi \rrbracket_{V[X/S]}$ is such that:

$$f(S) = f(\emptyset) \cup \bigcup_{x \in S} f(\{x\})$$

and thus f has a fixpoint with $\llbracket \mu(X = \varphi) \rrbracket_V = f^\infty(\emptyset)$.

PROOF. We will first prove by induction on the size of terms the following property: given a valid term φ , for all $S \neq \emptyset$ we have $\forall m \in \llbracket \varphi \rrbracket_{V[X/S]} \exists w_m \in S \ m \in \llbracket \varphi \rrbracket_{V[X/\{w_m\}]}$.

- Using lemma 1 the property is clearly true for terms φ such that X is not free in φ . And the only relation variable where X appears free is X . For X the property trivially holds (with $w_m = m$).
- For unary operators $\varphi \in \{\rho_a^b(\varphi_1), \tilde{\pi}_a(\varphi_1)\}$ we have $m \in \llbracket \varphi \rrbracket_{V[X/S]}$ implies the existence of $m' \in \llbracket \varphi_1 \rrbracket_{V[X/S]}$ such that m is the image of m' through this operator. By the induction hypothesis, for m' there is w such that $m' \in \llbracket \varphi_1 \rrbracket_{V[X/\{w_{m'}\}]}$ and thus $m \in \llbracket \varphi \rrbracket_{V[X/\{w_{m'}\}]}$.
- For a join operator $\varphi = \varphi_1 \bowtie \varphi_2$ we have by linearity that φ is constant in X for some i (let us note $\bar{i} = 3 - i$). If $\varphi_{\bar{i}}$ is also constant in X then φ is constant in X so we can refer to the first item. Otherwise, $m \in \llbracket \varphi \rrbracket_{V[X/S]}$ implies the existence of $m_1 \in \llbracket \varphi_1 \rrbracket_{V[X/S]}$ and $m_2 \in \llbracket \varphi_2 \rrbracket_{V[X/S]}$ such that $m = m_1 + m_2$. By induction, there exists w_{m_i} such that $m_i \in \llbracket \varphi_i \rrbracket_{V[X/\{w_{m_i}\}]}$. For i we have $\llbracket \varphi_i \rrbracket_{V[X/S]} = \llbracket \varphi_i \rrbracket_{V[X/\emptyset]} = \llbracket \varphi_i \rrbracket_{V[X/\{w_{m_i}\}]}$ which means that in any case $m_1 \in \llbracket \varphi_1 \rrbracket_{V[X/\{w_{m_1}\}]}$, $m_2 \in \llbracket \varphi_2 \rrbracket_{V[X/\{w_{m_2}\}]}$ and thus $m \in \llbracket \varphi \rrbracket_{V[X/\{w_{m_i}\}]}$.
- For the term $\varphi = \varphi_1 \triangleright \varphi_2$ with any mapping $m \in \llbracket \varphi \rrbracket_{V[X/S]}$ is built using at least one mapping m_1 from $\llbracket \varphi_1 \rrbracket_{V[X/S]}$. By induction, we have w such that $m_1 \in \llbracket \varphi_1 \rrbracket_{V[X/\{w\}]}$. But X does not appear free in φ_2 , thus $\llbracket \varphi_2 \rrbracket_{V[X/S]} = \llbracket \varphi_2 \rrbracket_{V[X/\{w\}]}$ and thus $\llbracket \varphi \rrbracket_{V[X/S]} = \llbracket \varphi \rrbracket_{V[X/\{w\}]}$.
- For the term $\varphi = \varphi_1 \cup \varphi_2$, $m \in \llbracket \varphi \rrbracket_{V[X/S]}$ implies $m \in \llbracket \varphi_1 \rrbracket_{V[X/S]}$ or $m \in \llbracket \varphi_2 \rrbracket_{V[X/S]}$. By induction we have w such that $m \in \llbracket \varphi_1 \rrbracket_{V[X/\{w\}]}$ or $m \in \llbracket \varphi_2 \rrbracket_{V[X/\{w\}]}$ and thus $m \in \llbracket \varphi \rrbracket_{V[X/\{w\}]}$.
- Given the term $\mu(Y = \varphi)$ we have $\mu(Y = \varphi)$ constant in X and thus the result by lemma 1.

□

LEMMA (1). Let φ be a term.

- If φ is recursive in X then for all V , $\llbracket \varphi \rrbracket_{V[X/\emptyset]} = \emptyset$.
- If φ is constant in X , then φ does not depend on X , i.e. for all S and V , $\llbracket \varphi \rrbracket_{V[X/S]} = \llbracket \varphi \rrbracket_{V[X/\emptyset]}$.

PROOF. For the constant part the result is trivial by induction.

For the recursive part, we also work by induction. It is true for base relations (constants cannot be recursive) but we need to be careful because the subterms of a recursive term can be non-recursive. By the definition of *rec* this can only happen for $\varphi_1 \bowtie \varphi_2$, but one of the φ_i has to be recursive and since the join with an empty set leads to an empty set the result holds by induction. □

PROPOSITION (2). A fixpoint term $\mu(X = \varphi)$, linear, positive and non mutually recursive can be rewritten to either: an empty term, a term φ with one less fixpoint or a decomposed fixpoint.

PROOF. Given a fixpoint $\mu(X = \varphi)$ we can always decompose φ into a Constant part C and a Recursive part R (possibly empty).

The idea is to prove by induction on φ that it is true for φ where X is linear, positive and non-mutually recursive:

- For a term φ constant in X the result is clear ($R = \emptyset, C = \varphi$).
- For X , $R = X, C = \emptyset$.
- For a unary operator $f(\varphi) \in \{\rho_a^b(\varphi), \tilde{\pi}_a(\varphi), \sigma_i(\varphi)\}$, we have φ that can be decomposed into R_φ, C_φ the solution is $f(R_\varphi), f(C_\varphi)$ (where $f(s)$ represents $f(s)$ if s is a term and $\emptyset$ otherwise).
- For a join $\varphi_1 \bowtie \varphi_2$, let us suppose by symmetry that φ_2 is constant in X ; then φ_1 can be decomposed into R, C and the result is $R \bowtie \varphi, C \bowtie \varphi$.
- For an antijoin, the same argument as the one for joins works.
- For unions the results for subterms can be merged.
- Fixpoints are constant in X by the non mutual recursion hypothesis.

□

PROPOSITION (3). *Given a database $(\mathcal{R}, \Gamma, D)$ and $\varphi \in \mathcal{F}[\Gamma]$, if $\Gamma \vdash \varphi : t$ then the relation $\llbracket \varphi \rrbracket_D$ has type t .*

PROOF. Let Γ be compatible with V . The property is thus true for relation variables; it is also true for constants and by induction unions, joins, antijoins, filters, duplication or removal of columns. This leaves us with fixpoints.

Suppose $\Gamma \vdash \mu(X = \varphi) : t$. The empty set of mappings is compatible with t , thus $\llbracket \varphi \rrbracket_{V[X/\emptyset]}$ is compatible with t by induction, and thus by further induction we have $\llbracket \mu(X = \varphi) \rrbracket_V$ compatible with t . □

B DATALOG & μ -RA EXPRESSIVE POWERS

In this section we present how to translate various Datalog into μ -RA. The results presented here are not at the heart of our work and most of them are already known in the literature (with very similar statements and with similar proofs, see e.g. [4] or [2] regarding Datalog and the while^+ language).

The only novelty of this proof relies in the proof that the linearity of *rest- μ -RA* actually reduces the expressive power. However to understand why we need to present a translation from Datalog to μ -RA and back. We will therefore not rely on formal proofs but we will build some intuition and provide examples.

B.1 Datalog with only one IBD

We recall in this section that datalog programs can always be transformed to programs that have only one recursive rule and one output rule (this is exercise 14.17 of the alic book [2]).

B.1.1 Step 1: the n -aryfication. Given a Datalog program P , we can always modify P so that all rules in P are n -ary for some n . To do that we simply take n to be the maximal arity over all the rules and extend all the rules with a constant c to match this arity.

For instance:

Path(1, 2).

Access(1).

Access(X) :- Access(Y), Path(X, Y)

can be made 3-ary in the following way:

Path(1, 2, c).

Access(1, c, c).

Access(X, c, c) :- Access(Y, c, c), Path(X, Y, c)

B.1.2 Step 2: one rule datalog. Given a Datalog program P , we can always modify P so that there is only one recursive rule and one “output” rule in P . The idea is to first convert P into a n -ary program P' (for some n) then creates a unique $n + 1$ rules that takes as its first argument the name of the rule. For instance, our running example becomes:

Rec(path, 1, 2, c).

Rec(access, 1, c, c).

Rec(access, X, c, c) :- Rec(access, Y, c, c), Rec(path, X, Y, c).

Output(X) :- Rec(access, X, c, c).

B.2 From a derivation rule to μ -RA

It is a well-known fact that non-recursive datalog and the relational algebra coincide (see e.g. chapter 14 of the alic book [2]). Given a production $\text{head}(\bar{Y}) : -\text{body}_1(\bar{X}_1), \dots, \text{body}_k(\bar{X}_k)$ we can translate $\text{body}_1(\bar{X}_1), \dots, \text{body}_k(\bar{X}_k)$ using $k - 1$ joins between each body_i , renames to rename arguments of body_i , antiprojections to remove existential variables, and filters for constants. Finally we use joins with constants for the constants of the head and renames for the variables.

For instance, if we translate the Datalog IBD *Rec* into a term *Rec* that has 4 columns (a_1, a_2, a_3 and a_4) the translation of the body

$\text{Rec}(\text{access}, X, c, c) :- \text{Rec}(\text{access}, Y, c, c), \text{Rec}(\text{path}, X, Y, c).$

is $\rho_{a_2}^Y(\tilde{\pi}_{a_1}(\tilde{\pi}_{a_3}(\tilde{\pi}_{a_4}(\sigma_{a_1=\text{access} \wedge a_3=c \wedge a_4=c}(\text{Rec})))) \bowtie \tilde{\pi}_{a_1}(\tilde{\pi}_{a_4}(\rho_{a_3}^Y(\rho_{a_2}^X(\sigma_{a_1=\text{path} \wedge a_4=c}(\text{Rec}))))))$

The whole translation is (using *body* to denote the above term):

$\rho_X^{a_2}(\tilde{\pi}_Y(\text{body})) \bowtie |a_3 \rightarrow c| \bowtie |a_4 \rightarrow c| \bowtie |a_1 \rightarrow \text{access}|$

B.3 From inflationary Datalog[⌊] to μ -RA

Given an inflationary-Datalog[⌊] program P that, w.l.o.g., has recursive rule Rec and one output rule $Output$ we can translate Rec to a fixpoint of the form $\mu(Rec = \varphi_1 \cup \dots \varphi_k)$ where each φ_i corresponds to one derivation of the rule Rec . Finally we translate each production of $Output$ into a term ψ (where Rec is replaced by the fixpoint above) and we generate a term that is the union of all these ψ .

Given our initial example we have the term O (here we cut the translation to ease the reading):

$$\begin{aligned}
B_1 &= |a_1 \rightarrow path| \bowtie |a_2 \rightarrow 1| \bowtie |a_3 \rightarrow 2| \bowtie |a_4 \rightarrow c| \\
B_2 &= |a_1 \rightarrow access| \bowtie |a_2 \rightarrow 1| \bowtie |a_3 \rightarrow c| \bowtie |a_4 \rightarrow c| \\
B_3 &= \rho_X^{a_3}(\tilde{\pi}_Y(\rho_{a_2}^Y(\tilde{\pi}_{a_1}(\tilde{\pi}_{a_3}(\tilde{\pi}_{a_4}(\sigma_{a_1=access \wedge a_3=c \wedge a_4=c}(Rec)))))) \bowtie \rho_{a_3}^Y(\rho_{a_2}^X(\tilde{\pi}_{a_1}(\tilde{\pi}_{a_4}(\sigma_{a_1=path \wedge a_4=c}(Rec)))))) \\
B_4 &= \mu(Rec = B_1 \cup B_2 \cup B_3) \\
O &= \tilde{\pi}_{a_1}(\tilde{\pi}_{a_3}(\tilde{\pi}_{a_4}(\sigma_{a_1=access}(B)_4)))
\end{aligned}$$

The semantics does coincide with inflationary-Datalog[⌊] because the formula $B_1 \cup B_2 \cup B_3$ captures the “immediate consequence” of the Datalog program.

B.4 From stratified Datalog to μ -RA

In a stratified Datalog program, each rule can be indexed with an integer n such that a negation of a rule indexed by k can only appear in the production rule of a term indexed with $k' > k$.

In the case of a stratified Datalog program, merging all the rules into one will break the stratification. The trick here is to operate stratum by stratum and translate the stratum i into a rule Rec_i . The resulting program will have one rule per stratum.

Just like in the inflationary case, each stratum i can be translated into a unique fixpoint $\mu(X_i = \varphi_i)$. The production rules of the stratum i can only reference to a Rec_j where $j \leq i$. We translate Rec_i into X_i and the Rec_j into $\mu(X_j = \varphi_j)$. Note that each φ_i can contain several occurrences of Rec_j with $j < i$ and that makes the translation exponential but all the fixpoints do are non mutually recursive and positive.

Let us consider the following example (already stratified):

Path(...) an EDB

Access_1($\emptyset$).

Access_1(X) :- Access_1(Y), Path(Y, X)

Access_2(1).

Access_2(X) :- Access_2(Y), Path(Y, X), not Access_1(Y)

We translate Path into a term $\mu(X_0 = \varphi_0)$ (despite the fixpoint φ_0 is actually not recursive as Path is an EDB). Then we translate Access₁:

$$\mu(X_1 = |a_1 \rightarrow 0| \cup \rho_{a_2}^{a_1}(\tilde{\pi}_{a_1}(X_1 \bowtie Path)))$$

Then we translate Access₂ (using Access₁ to denote the term above) :

$$\mu(X_2 = |a_1 \rightarrow 1| \cup \rho_{a_2}^{a_1}(\tilde{\pi}_{a_1}(X_2 \bowtie Path \triangleright Access_1)))$$

B.5 From linear Datalog to *rest*- μ -RA

Given a linear Datalog program, we can use the stratified translation. In the resulting term each φ_i is composed of $\psi_1 \cup \dots \psi_k$ where each of the ψ_j corresponds to a linear production rule and thus contains at most one occurrence of X_i therefore our μ -RA term is also linear (in addition to be recursive and positive as proved by the stratified translation). All in all, our term does belong to *rest*- μ -RA.

B.6 From *rest*- μ -RA to linear Datalog

This direction is actually very simple once we know how to translate a term to a Datalog program, we just need to check that the resulting term is actually linear. To translate terms into Datalog, we work bottom-up associating each subterm φ to a Datalog rule. Datalog rules have columns that are indexed (there is a first column, a second, a third, etc.) while μ -RA has

column names. To handle this discrepancy, we suppose that we have calculated the type of each term (i.e. we compute a set of column names), then we order column names (any total order on the column names can be used).

The only difficulty here is the language of filters, in *rest-μ-RA* we actually impose no restriction on the filter conditions; for the translation we suppose that only the equality is used.

We thus recursively create production rules for each Datalog predicate $s_\varphi(\bar{T})$ corresponding the each term $\varphi s_\varphi(\bar{T})$ (where $\bar{T}$ is the ordered set of columns of the type of φ).

- For $\varphi = \varphi_1 \bowtie \varphi_2$ we create a rule for the join: $s_\varphi(\bar{T}) \leftarrow s_{\varphi_1}(\bar{T}_1), s_{\varphi_2}(\bar{T}_2)$.
- For $\varphi = \varphi_1 \cup \varphi_2$ we have two production rules, one for each φ_i : $s_\varphi(\bar{T}) \leftarrow s_{\varphi_i}(\bar{T}_i)$.
- For $\varphi = \varphi_1 \triangleright \varphi_2$ we create the rule $s_\varphi \leftarrow s_{\varphi_1}(\bar{T}_1), \neg s_{\varphi_2}(\bar{T}_2)$.
- For $\varphi = \sigma_{a=b}(\varphi')$ we create the rule $s_\varphi(\bar{T}_1, b, \bar{T}_2) \leftarrow s_{\varphi'}(\bar{T}_1, b, \bar{T}_2)$ if we suppose that the ordered type of φ' is $\bar{T}_1, a, \bar{T}_2$
- For $\varphi = \pi_p(\varphi')$ we create the rule $s_\varphi(\bar{T}_\varphi) \leftarrow s_{\varphi'}(\bar{T}_{\varphi'})$
- For $\varphi = \rho_a^b(\varphi')$ we create the rule $s_\varphi(\bar{T}') \leftarrow s_{\varphi'}(\bar{T}_{\varphi'})$ where $\bar{T}'$ is $\bar{T}_{\varphi'}$ where we inserted a a in the place of where b will be stored.
- For $\varphi = \mu(X = \varphi')$ we create the rule $s_X(\bar{T}) \leftarrow s_{\varphi'}(\bar{T}_{\varphi'})$.
- For $\varphi = X$ we create the rule $s_X(\bar{T}) \leftarrow s_{\varphi'}(\bar{T}_{\varphi'})$.

Since the *rest-μ-RA* term is linear we can see that each production rule contain at most one subgoal that is recursive with the head.

C PROOFS FOR SECTION 4 (GENERATING NEW QUERY PLANS)

LEMMA (2). *Let w be a mapping and φ a term linear, positive and non mutually recursive in X . For all $m \in \llbracket \varphi \rrbracket_{V[X/\{w\}]}$ either $m \in \llbracket \varphi \rrbracket_{V[X/\emptyset]}$ or there exists $p \in d(\varphi, X)$ such that for all $c \in \text{dom}(w)$:*

$$\left(p(c) = \perp \right) \vee \left(p(c) \notin \text{dom}(w) \right) \vee \left(m(c) = w(p(c)) \right)$$

PROOF. Let w and $m \in \llbracket \varphi \rrbracket_{V[X/\{w\}]} \setminus \llbracket \varphi \rrbracket_{V[X/\emptyset]}$. By induction:

- Since m exists, X can only be free in φ (no $|c \rightarrow v|$, no $Y \neq X$, no fixpoints).
- For a relation X , the result is clear.
- For a union we have i such that $m \in \llbracket \varphi_i \rrbracket_{V[X/\{w\}]}$ and thus the result.
- For a join $\varphi_1 \bowtie \varphi_2$ let us suppose by symmetry that φ_1 is not constant in X and that φ_2 is. We have $m_1 \in \llbracket \varphi \rrbracket_{V[X/\{w\}]}$, $d_1 \in d(\varphi_1, X) \subseteq d(\varphi_1 \bowtie \varphi_2, X)$ (with inductive hypothesis) and $m_2 \in \llbracket \varphi \rrbracket_{V[X/\emptyset]}$. For each $c \in \text{dom}(w)$ we either have $d_1(c) = \perp$ or $d_1(c) \notin \text{dom}(w)$ or $m_1(c) = w(d_1(c))$ and thus $m(c) = w(d_1(c))$. Note that $m_2(c)$ might or might not be defined but if $(d_1(c) \neq \perp) \wedge (p(c) \in \text{dom}(w))$ then $m_1(c)$ is also defined and $m(c) = m_1(c)$.
- For an antijoin or a filter the result is clear.
- For a column rename or removal, the definition of d makes it work. Let us note $\lambda(\varphi)$ the term, we have $m \in \llbracket \lambda(\varphi) \rrbracket_{V[X/\{w\}]}$ that implies $m' \in \llbracket \varphi \rrbracket_{V[X/\{w\}]}$ and $d' \in d(\varphi, X)$ with the property. And $d' \circ \lambda$ works.

□

LEMMA (3). *Given a fixpoint term $\mu(X = \varphi) \in \mathcal{F}[\Gamma]$ of type t and a mapping w of type t , $w \in \llbracket \mu(X = \varphi) \rrbracket_V$ if and only if there exists a lineage for w , that is, a finite sequence $w_0, \dots, w_n$ such that: $w_0 \in \llbracket \varphi \rrbracket_{V[X/\emptyset]}$, $w_{i+1} \in \llbracket \varphi \rrbracket_{V[X/\{w_i\}]}$, and $w_n = w$.*

Furthermore, for all lineages $w_0, \dots, w_n$ and all $c \in t \cap \text{stab}(\varphi, X)$, we have for all i , $w_0(c) = w_i(c)$.

PROOF. Let $w \in \llbracket \mu(X = \varphi) \rrbracket_V$ and let n minimal such that $w \in U_n$ (as defined by the semantic). By iterating proposition 1 we find $w_0, \dots, w_n = w$ as expected.

Conversely if we have such $w_0, \dots, w_n = w$ then clearly $w \in \llbracket \mu(X = \varphi) \rrbracket_V$.

Now, by Lemma 2, for each $0 \leq i \leq n-1$, the mappings w_i and w_{i+1} there is $p \in d(\varphi, X)$ such that for all $c \in \text{stab}(\varphi, X) \cap t$, $w_{i+1}(c) = w_i(p(c)) = w_i(c)$. By iteration so does w_0 and w . □

THEOREM (1 PUSHING FILTERS). *Let $\mu(X = \kappa \cup \psi)$ be a decomposed fixpoint term, V an environment and $\dagger$ a filter condition with $FC(\dagger) \subseteq \text{stab}(\psi, X)$. Then we have $\llbracket \sigma_{\dagger}(\mu(X = \kappa \cup \psi)) \rrbracket_V = \llbracket \mu(X = \sigma_{\dagger}(\kappa) \cup \psi) \rrbracket_V$.*

PROOF. Let $w \in \llbracket \sigma_{\bar{f}}(\mu(X = \kappa \cup \psi)) \rrbracket_V$. Let $w_0, \dots, w_n$ be a lineage of w : w passes the filter and by Lemma 3, w has the same values as all the w_i on $FC(\bar{f})$; therefore w_0 also passes the filter: we have $w_0 \in \llbracket \sigma_{\bar{f}}(\kappa) \rrbracket_V$. Thus $w_0, \dots, w_n$ is also a lineage of $\llbracket \mu(X = \sigma_{\bar{f}}(\kappa) \cup \psi) \rrbracket_V$.

Conversely, if $w \in \llbracket \mu(X = \sigma_{\bar{f}}(\kappa) \cup \psi) \rrbracket_V$, let $w_0, \dots, w_n$ be a lineage of w ; we have $w_0 \in \llbracket \sigma_{\bar{f}}(\kappa) \rrbracket_V$, thus w_0 passes the filter and by the same argument as above, so must w . $\square$

THEOREM (2 PUSHING ANTI-JOINS). *Let $\mu(X = \kappa \cup \psi)$ be a decomposed fixpoint term, V an environment and ξ a term of type $t \subseteq \text{stab}(\psi, X)$ (we suppose that X is not a free variable of ξ). Then we have $\llbracket \mu(X = \kappa \cup \psi) \triangleright \xi \rrbracket_V = \llbracket \mu(X = (\kappa \triangleright \xi) \cup \psi) \rrbracket_V$.*

LEMMA (4). *Let $\mu(X = \kappa \cup \psi) \in \mathcal{F}[\Gamma]$ be a decomposed fixpoint of type t , let $c \in (\mathcal{C} \setminus t)$ that can be added to ψ , and w a mapping of type t . We note $w(v) = w \cup \{c \rightarrow v\}$.*

If $\forall R \in \mathcal{R}, c \notin \Gamma(R)$, then we have:

- $c \in \text{stab}(\psi, X)$
- $\Gamma \cup \{X \rightarrow t \cup \{c\}\} \vdash \psi : t \cup \{c\}$
- $\llbracket \psi \bowtie |c \rightarrow v| \rrbracket_{V[X/\{w\}]} = \llbracket \psi \rrbracket_{V[X/\{w(v)\}]}$

PROOF. We will prove the result $\llbracket \psi \rrbracket_{w(v)} = \llbracket \psi \bowtie |c \rightarrow v| \rrbracket_w$ inductively on the size of ψ a term recursive in X .

Note that when a subformula ξ of ψ is constant in X we have that $\llbracket \xi \rrbracket_{V[X/\{w\}]} = \llbracket \xi \rrbracket_{V[X/\{w(v)\}]}$ by lemma 1 and we also have that c is not in the type of this ξ (since $\forall R, c \notin \Gamma R$ and the definition of *add* forbids to rename a column into c). Note also that subformula that are fixpoints or constants are necessarily constant in X .

Let us explore the various cases. For the simplicity of proofs, we use $\bowtie$ and $\triangleright$ directly with sets of mappings (e.g. $A \bowtie B = \{m_A + m_B \mid m_A \in A, m_B \in B \wedge m_A \sim m_B\}$).

- For the formula X , the result is trivial and it is the only base case (constants and other variables cannot be recursive in X).
- For $\varphi_1 \bowtie \varphi_2$, one of φ_1, φ_2 has to be constant, the other recursive. By symmetry, we suppose that φ_1 is recursive and φ_2 constant. We have $\llbracket \varphi_1 \bowtie \varphi_2 \rrbracket_{V[X/\{w(v)\}]} = \llbracket \varphi_1 \rrbracket_{V[X/\{w(v)\}]} \bowtie \llbracket \varphi_2 \rrbracket_{V[X/\{w(v)\}]} = \llbracket \varphi_1 \bowtie |c \rightarrow v| \rrbracket_{V[X/\{w\}]} \bowtie \llbracket \varphi_2 \rrbracket_{V[X/\{w\}]} = \llbracket (\varphi_1 \bowtie |c \rightarrow v|) \bowtie \varphi_2 \rrbracket_{V[X/\{w\}]} = \llbracket (\varphi_1 \bowtie \varphi_2) \bowtie |c \rightarrow v| \rrbracket_{V[X/\{w\}]}$
- For $\varphi_1 \triangleright \varphi_2$ we similarly have $\llbracket \varphi_1 \triangleright \varphi_2 \rrbracket_{V[X/\{w(v)\}]} = \llbracket \varphi_1 \rrbracket_{V[X/\{w(v)\}]} \triangleright \llbracket \varphi_2 \rrbracket_{V[X/\{w(v)\}]} = \llbracket \varphi_1 \bowtie |c \rightarrow v| \rrbracket_{V[X/\{w\}]} \triangleright \llbracket \varphi_2 \rrbracket_{V[X/\{w\}]} = \llbracket (\varphi_1 \bowtie |c \rightarrow v|) \triangleright \varphi_2 \rrbracket_{V[X/\{w\}]} = \llbracket (\varphi_1 \triangleright \varphi_2) \bowtie |c \rightarrow v| \rrbracket_{V[X/\{w\}]}$ and the last line that uses the commutativity of $\triangleright$ over $\bowtie$ is only true because c cannot be in the type of φ_2 .
- For $\sigma_{\bar{f}}(\varphi')$, $\pi_a(\varphi')$ and $\rho_a^b(\varphi)$ the result comes easily as $c \notin \{a, b\} \cup FC(f)$.

$\square$

THEOREM (3 PUSHING JOINS). *Let $\mu(X = \kappa \cup \psi) \in \mathcal{F}[\Gamma]$ be a decomposed fixpoint of type t_κ and $\varphi \in \mathcal{F}[\Gamma]$ (with $X \notin \text{free}(\varphi)$) a term of type t_φ such that:*

- (1) $t_\varphi \subseteq \text{stab}(\psi, X)$
- (2) $\forall c \in t_\varphi \setminus t_\kappa \text{ add}(\psi, X, c)$

Then we have $\Gamma \vdash \mu(X = \kappa \bowtie \varphi \cup \psi) : t_\varphi \cup t_\kappa$ with for all V compatible with Γ :

$$\llbracket \varphi \bowtie \mu(X = \kappa \cup \psi) \rrbracket_V = \llbracket \mu(X = \kappa \bowtie \varphi \cup \psi) \rrbracket_V$$

PROOF. Lemma 4 ensures us that $\Gamma \cup \{X \rightarrow t_\varphi \cup t_\kappa\} \vdash \psi : t_\varphi \cup t_\kappa$, and thus $\Gamma \vdash \mu(X = \varphi \bowtie \kappa \cup \psi) : t_\varphi \cup t_\kappa$.

Then if we take a lineage $w_0 \dots w_n$ of $\llbracket \mu(X = \kappa \cup \psi) \rrbracket_V$ and there exists $u \in \llbracket \varphi \rrbracket_V$ compatible with w_n then $t_\varphi \subseteq \text{stab}(\psi, X)$ ensures us that u is compatible with all w_i .

Then by iterating Lemma 4, for each i and for each $c \in t_\varphi \setminus t_\kappa$, we have that $w_0(u), \dots, w_n(u)$ is a valid lineage of $\llbracket \mu(X = \kappa \bowtie \varphi \cup \psi) \rrbracket_V$ and reciprocally. $\square$

THEOREM (4 MERGING FIXPOINTS). *Given two decomposed fixpoints $\mu(X = \kappa_1 \cup \psi_1)$ and $\mu(X = \kappa_2 \cup \psi_2)$ of types t_1 and t_2 such that:*

- (1) $t_1 \cap t_2 \subseteq \text{stab}(\psi_2, X, C_2) \cap \text{stab}(\psi_1, X, C_1)$
- (2) $\forall c \in t_1 \setminus t_2 \text{ add}(\psi_2, X, c)$
- (3) $\forall c \in t_2 \setminus t_1 \text{ add}(\psi_1, X, c)$

then we have:

$$\llbracket \mu(X = \kappa_1 \bowtie \kappa_2 \cup \psi_1 \cup \psi_2) \rrbracket_V.$$

$$\llbracket \mu(X = \kappa_1 \cup \psi_1) \bowtie \mu(X = \kappa_2 \cup \psi_2) \rrbracket_V =$$

PROOF. For $i \in \{1, 2\}$, let $w_0, \dots, w_{n_i}$ be a lineage of $\llbracket \mu(X = \kappa^i \cup \psi^i) \rrbracket_V$ with w_{n_1} compatible with w_{n_2} ; we can easily construct a lineage of size $n_1 + n_2$ of the form $(w_0^1 + w_0^2) \dots (w_{n_1}^1 + w_0^2) \dots (w_{n_1}^1 + w_{n_2}^2)$ and for the same reason as the last theorem, it holds.

Now let us take a lineage $w_0, \dots, w_n$ of $\llbracket \mu(X = \kappa_1 \bowtie \kappa_2 \cup \psi_1 \cup \psi_2) \rrbracket_V$. We decompose w_i into $w_i^1 + w_i^2$ where w_i^j is the restriction of w_i to the type of κ_j . Those w_i^j are not necessarily forming a lineage but we consider the subsequence containing w_0^i and for each $i > 0$ w_j^i when $w_i^j \in \llbracket \psi_i \rrbracket_{V[X/\{w_{i-1}^j\}]}$. Then by the theorem condition when $w_i^j \in \llbracket \psi_i \rrbracket_{V[X/\{w_{i-1}^j\}]}$ we have $w_i^j = w_{i-1}^j$. The two resulting sequences are thus lineages and we have the expected theorem. $\square$

THEOREM (5 PUSHING ANTIPROJECTIONS). *Let $\mu(X = \kappa \cup \psi) \in \mathcal{F}[\Gamma]$ be a decomposed fixpoint of type t_κ . Let $b \in \mathcal{C}$ be such that $\text{add}(\psi, X, b)$. Then:*

$$\llbracket \tilde{\pi}_b(\mu(X = \kappa \cup \psi)) \rrbracket_V = \llbracket \mu(X = \tilde{\pi}_b(\kappa) \cup \psi) \rrbracket_V$$

PROOF. This is a conclusion of lemma 4. Let $w_0, \dots, w_n$ be a lineage of $\llbracket \mu(X = \tilde{\pi}_c(\kappa) \cup \psi) \rrbracket_V$ there exists v such that $w_0(v) \in \llbracket \kappa \rrbracket_{V[X/\emptyset]}$ and if we have $w_i(v)$ we can find $w_{i+1}(v) \in \llbracket \psi \rrbracket_{V[X/\{w_i(v)\}]}$ by lemma 4. In the end we have a lineage for $w(v) \in \llbracket \mu(X = \kappa \cup \psi) \rrbracket_V$ and which means $w \in \tilde{\pi}_c(\mu(X = \kappa \cup \psi))$.

Notice that lemma 4 gives an equality therefore this is a bijection between lineage and also proves the converse way. $\square$

REFERENCES FOR THE APPENDIX

- [2] Serge Abiteboul, Richard Hull, and Victor Vianu, editors. *Foundations of Databases: The Logical Level*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1st edition, 1995.
- [4] Serge Abiteboul and Victor Vianu. Datalog extensions for database queries and updates. *J. Comput. Syst. Sci.*, 43(1):62–124, August 1991.